

第三讲 定时器，PWM与电机驱动

责任助教：赵欣然 王易科



01 定时器



定时器 (Timer) 的功能

- 定时器 (Timer) 主要功能：**计时**
 - 一些任务需要**每隔一定周期**执行一次
 - **PWM (脉冲宽度调制)** 输出信号具有**周期性**
 - 有对**信号出现次数**进行统计的需求
 -
- 功能实现：通过**计数**来实现**计时**

定时器工作流程

以LEDC (LED PWM控制器) 专用定时器产生LEDPWM为例

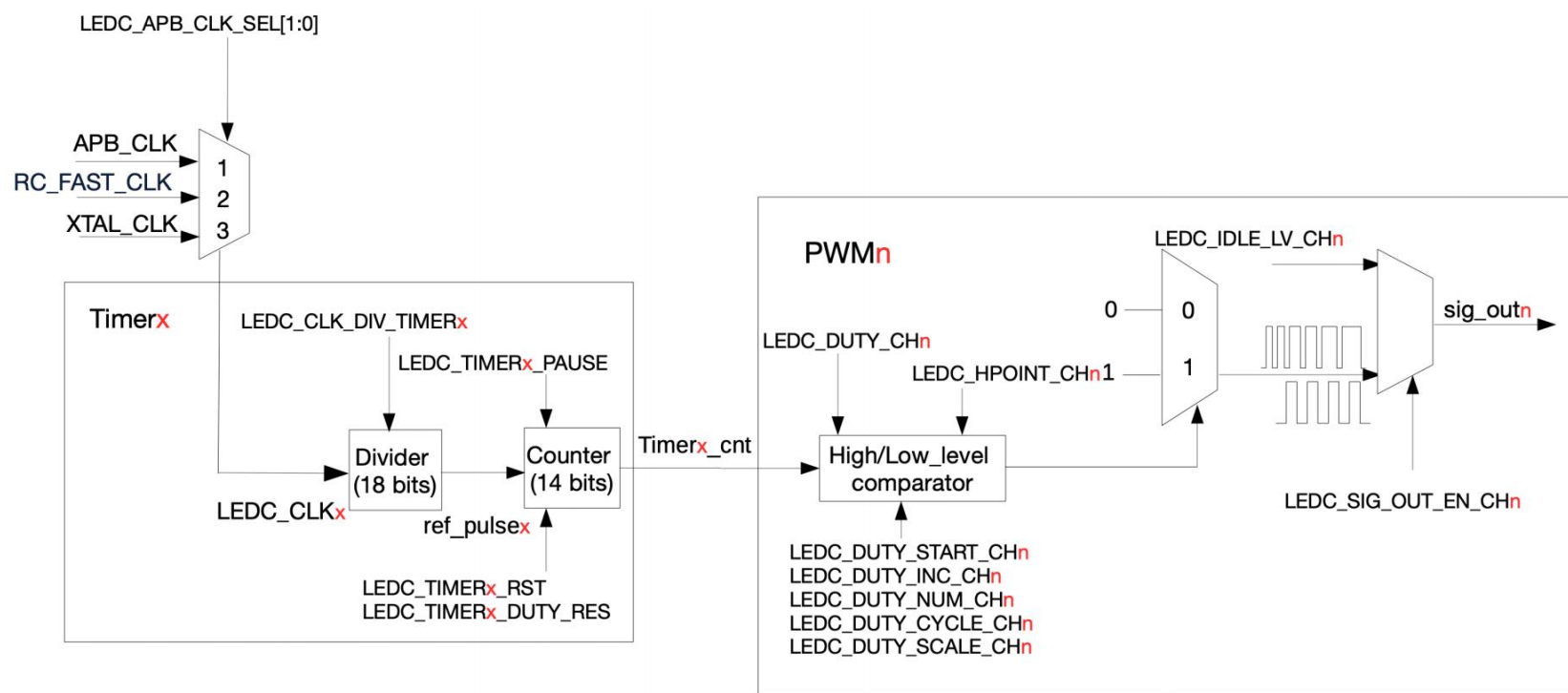


图 35.2-1. 定时器和 PWM 生成器功能块

定时器工作流程

以LEDC专用定时器产生LED PWM为例

时钟源

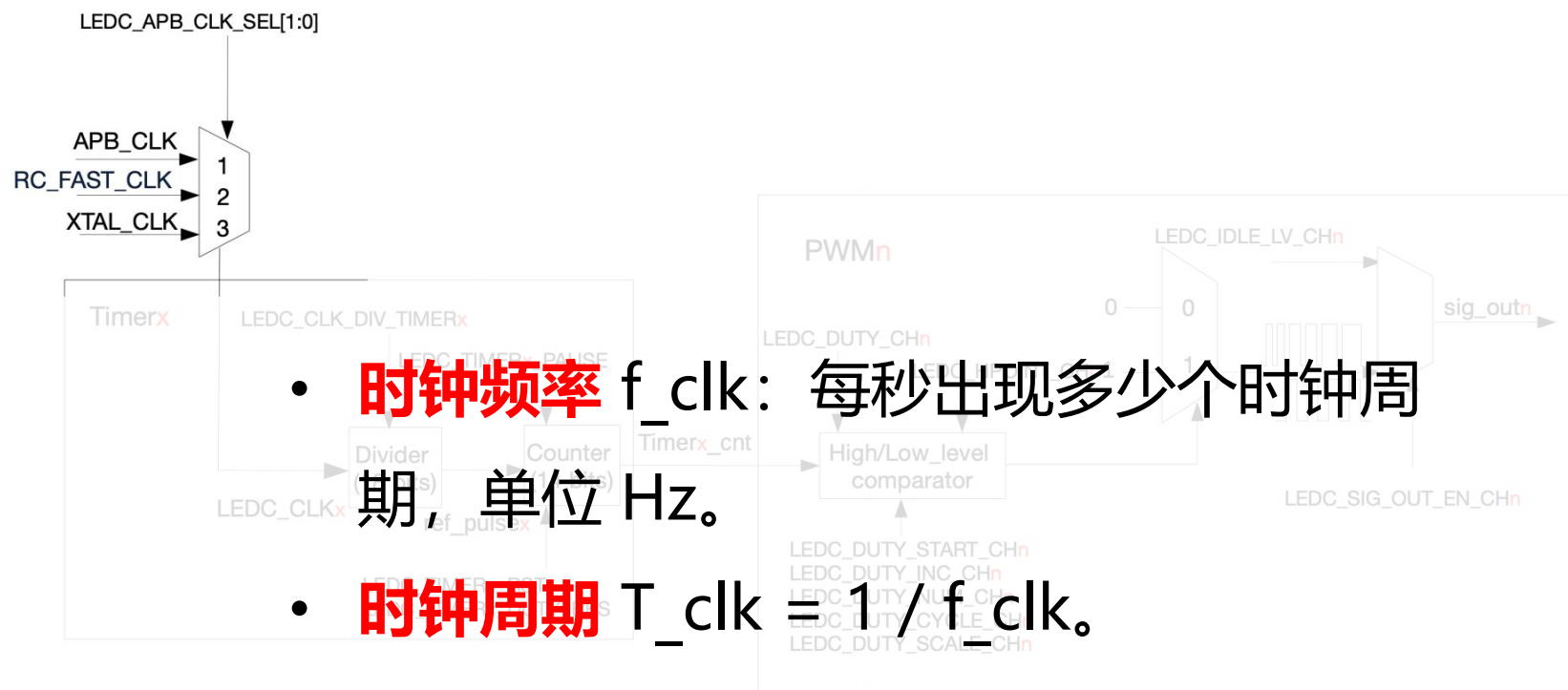


图 35.2-1. 定时器和 PWM 生成器功能块

定时器工作流程

以LEDC专用定时器产生LEDPWM为例

时钟源

- 数字芯片的所有**时序操作**都依赖**时钟**驱动，时钟频率直接决定运算速度、定时精度和功耗，**不同外设**对**时钟频率**的需求**差异极大**
- ESP32-S3 的时钟主要来源于**振荡器**、**RC 振荡电路**和**PLL** 时钟生成电路。

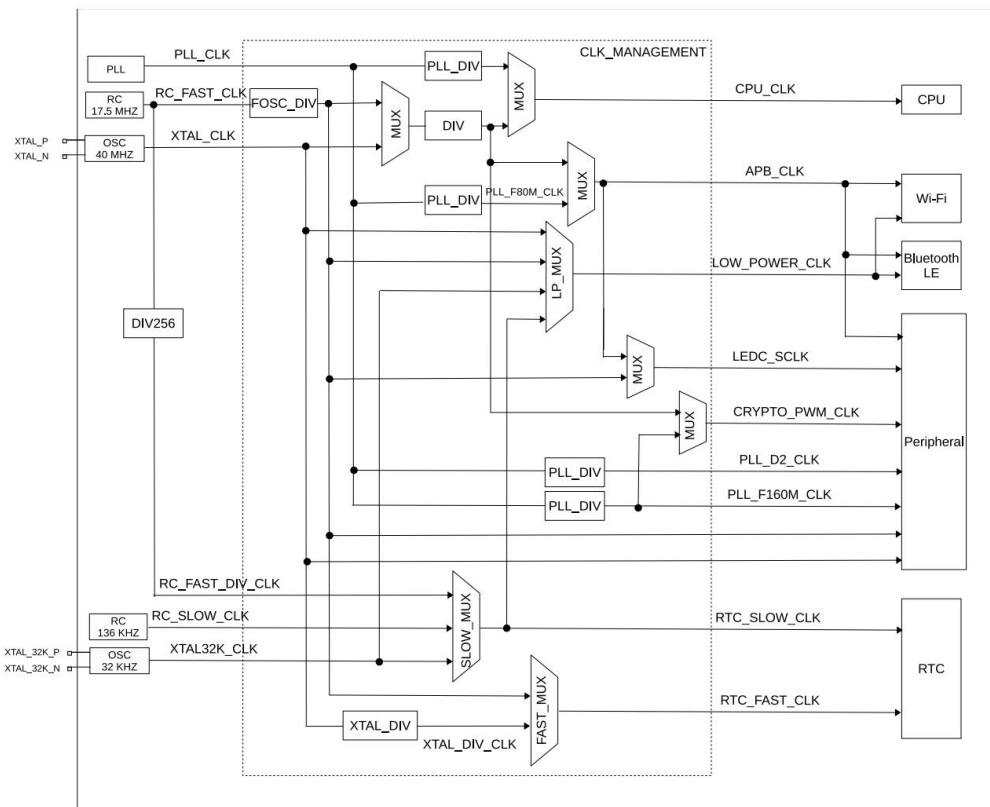


图 7.2-1. 系统时钟

定时器工作流程

以LEDC专用定时器产生LEDPWM为例

时钟源 PLL (Phase Locked Loop) 锁相环

- 功能：**倍频**，将低频基准时钟提升为高频系统时钟
- 结构与原理：

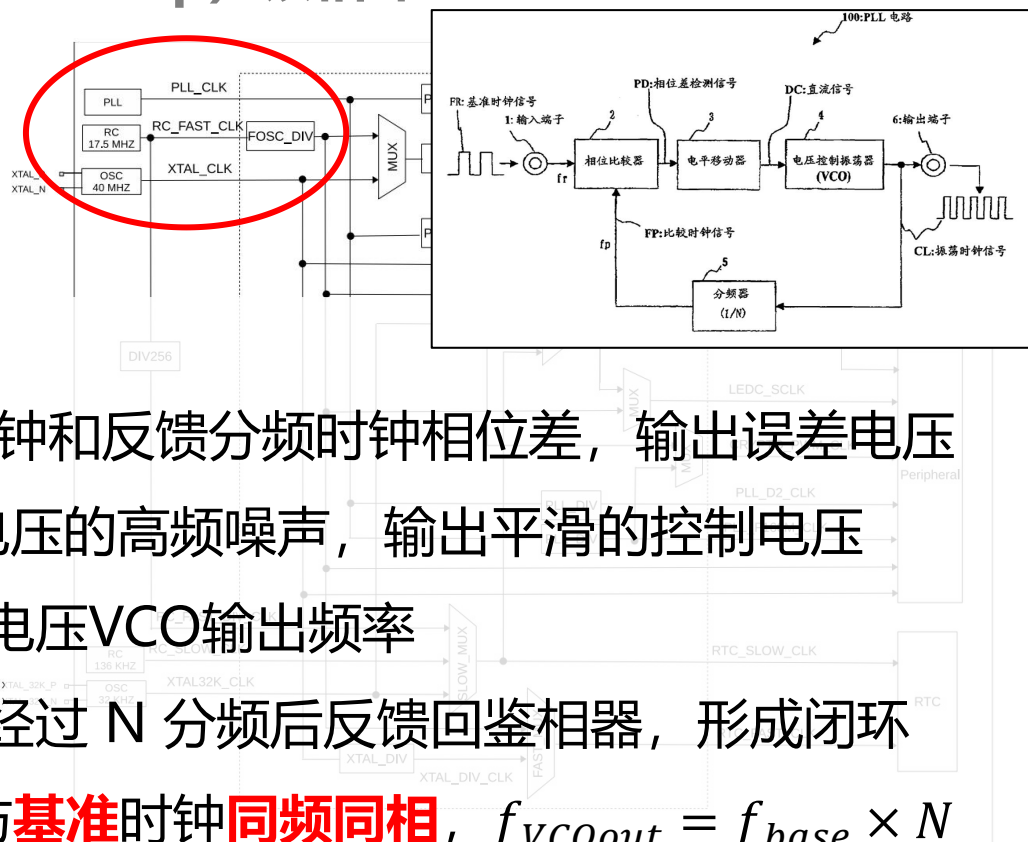


图 7.2-1. 系统时钟

- 鉴相器 PD: 比较输入时钟和反馈分频时钟相位差, 输出误差电压
- 环路滤波器 LPF: 误差电压的高频噪声, 输出平滑的控制电压
- 压控振荡器 VCO: 控制电压VCO输出频率
- 反馈分频器: VCO 输出经过 N 分频后反馈回鉴相器, 形成闭环
- 环路锁定后, **反馈时钟与基准时钟同频同相**, $f_{VCOout} = f_{base} \times N$

定时器工作流程

以LEDC专用定时器产生LEDPWM为例

时钟源

- 时钟经时钟**分频器**或时钟**选择器**等时钟模块的处理，使得大部分功能模块可以根据不同功耗和性能需求来获取及**选择对应频率的工作时钟**

e.g.: EPS32-S3中以外部40MHz无源晶振 (XTAL_CLK) 为基准，通过PLL生成最高480MHz的高频主时钟，在经过分频后为CPU（最高240MHz）、高速总线、高速外设提供时钟源

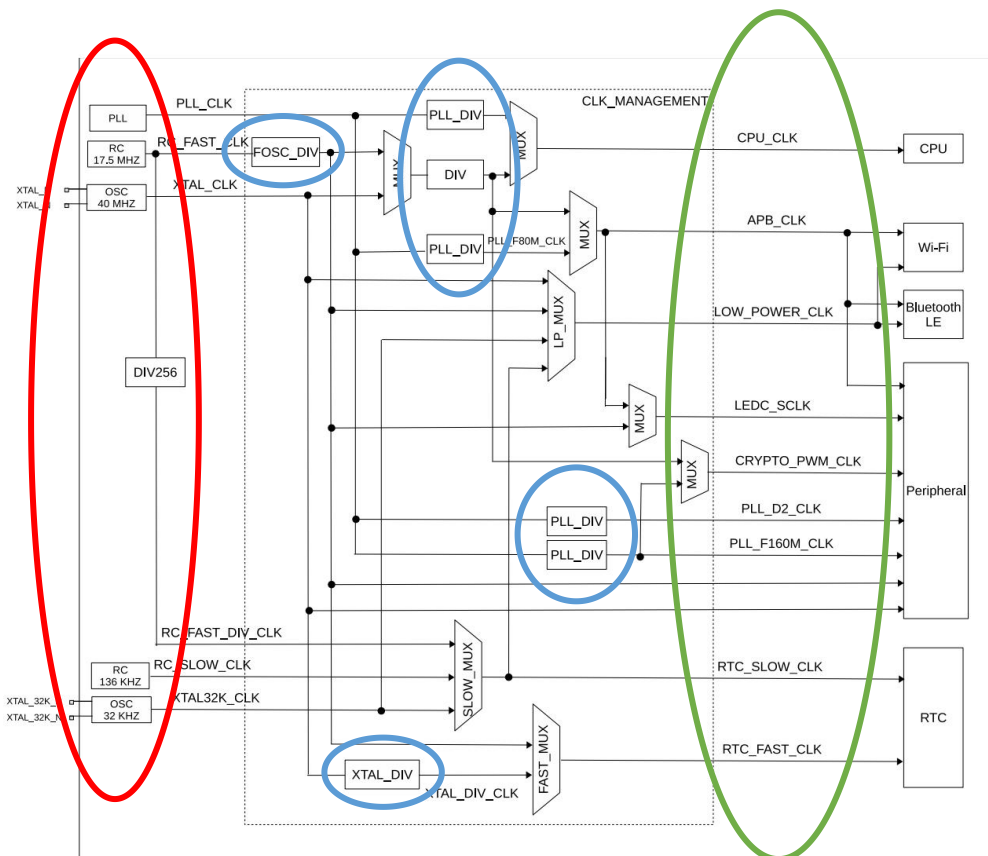


图 7.2-1. 系统时钟

定时器工作流程

以LEDC专用定时器产生LEDPWM为例

时钟源—预分频器

- 将输入的时钟信号进行**分频**，以产生**适合的时基时钟** (TB_CLK) 供时基计数器使用。ESP32 的定时器支持配置分频系数，范围从 2 到 65536，可以对时钟进行灵活分频。
- 若**输入频率**为 f_{src} ，整数**分频系数**为 N ：

$$f_{tick} = f_{src} / N$$

$$T_{tick} = N / f_{src}$$

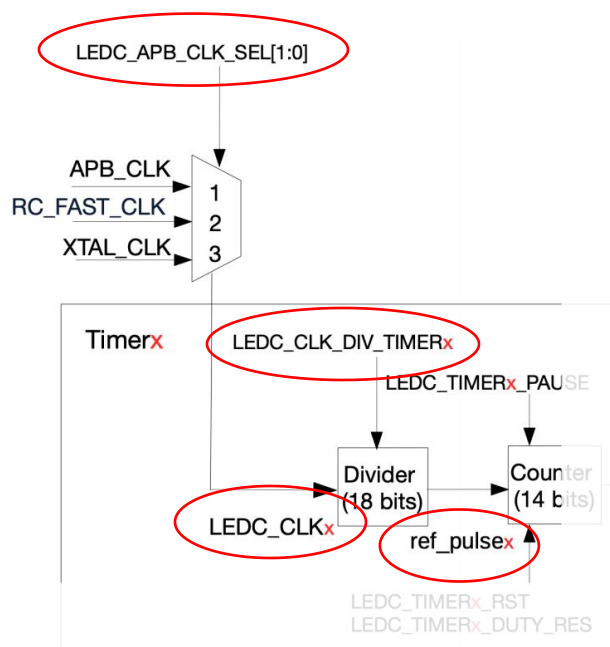
- *e.g. : 假设输入为 80 MHz, 取 $N = 80$, 则 $f_{tick} = 1 \text{ MHz}$, 一个 tick 为 $1 \mu s$ 。*

定时器工作流程

以LEDC专用定时器产生LEDPWM为例

时钟源—预分频器

若输入频率为 f_{src} ，整数分频系数为 N : $f_{tick} = f_{src} / N$



- 通过配置LEDC_APB_CLK_SEL[1:0]为LEDC_CLKx选择时钟源信号进入分频器
- LEDC_CLKx信号频率经分频系数LEDC_CLK_DIV分频后，产生ref_pulsex信号供计数器使用

图 35.2-1. 定时器和 PWM 生成器功能块

定时器工作流程

以LEDC专用定时器产生LEDPWM为例

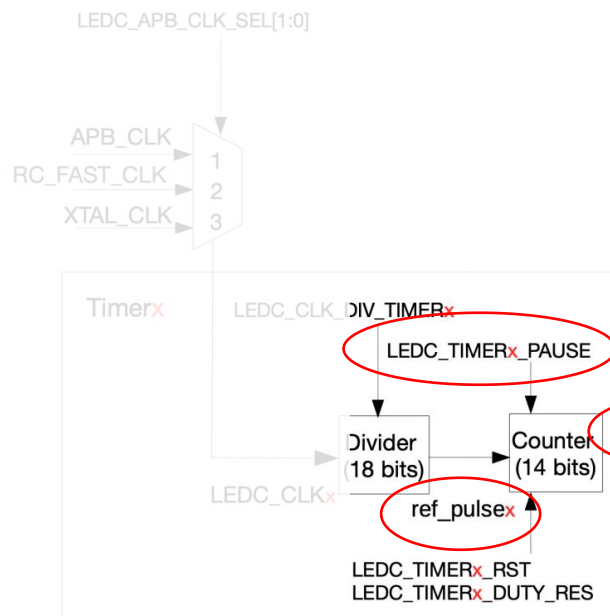
时钟源—预分频器—计数器

- 计数器可以**向上或向下计数**，可以通过寄存器 TIMG_Tx_INCREASE 来控制计数器的增减方向。
- **W 位**无符号计数器可表示 **$0 \sim 2^W - 1$** 。

定时器工作流程

以LEDC专用定时器产生LEDPWM为例

时钟源—预分频器—计数器



- LEDC中每个定时器有一个以 `ref_pulsex` 为**基准时钟**的14 位时基计数器。
- `LEDC_TIMERx_DUTY_RES` 字段用于配置14 位计数器的最大值。因此，PWM 信号的最大精确度为**14 位**。计数器最大可计数至 $2^{\text{LEDC_TIMERx_DUTY_RES}} - 1$ ，然后溢出重新从 0 开始计数。软件可以读取、复位、暂停计数器

图 35.2-1. 定时器和 PWM 生成器功能块

定时器工作流程

以LEDC专用定时器产生LEDPWM为例

时钟源—预分频器—计数器—比较器

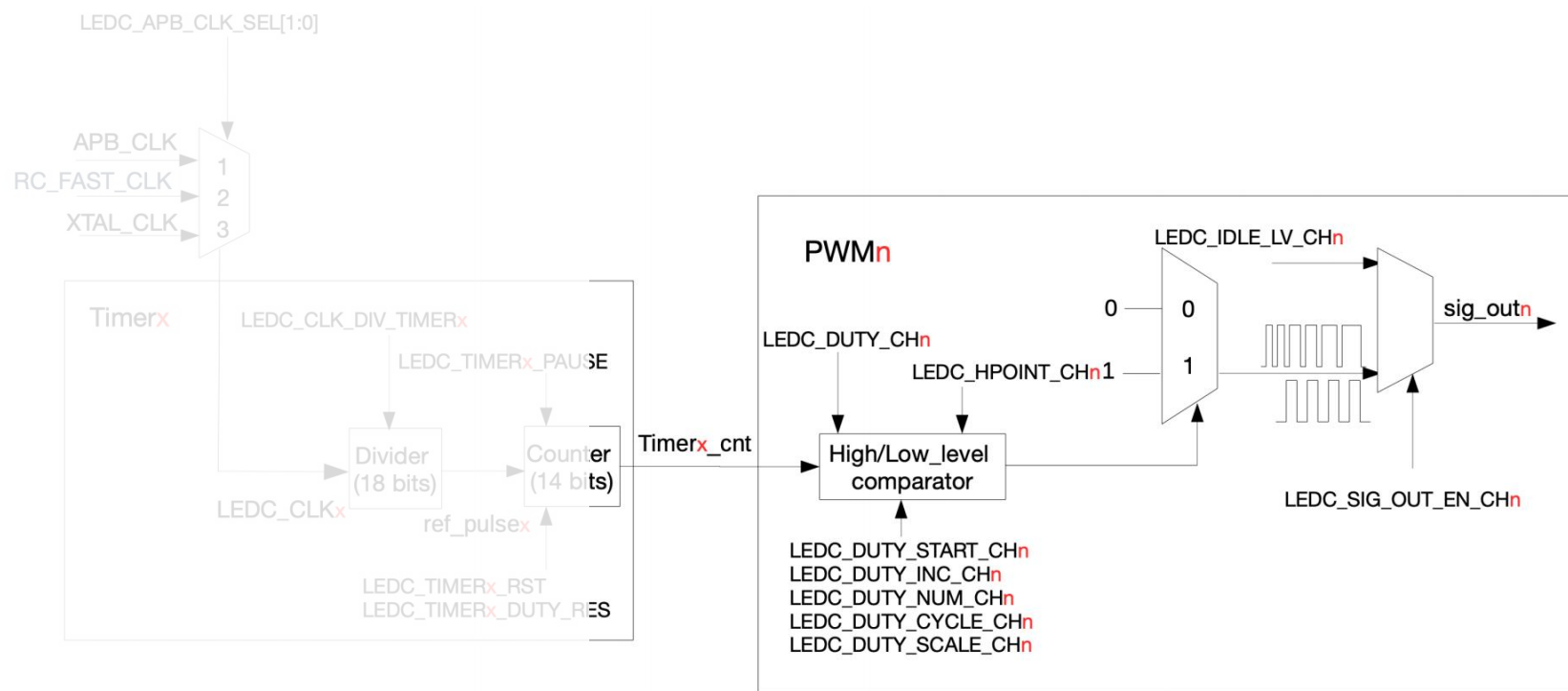


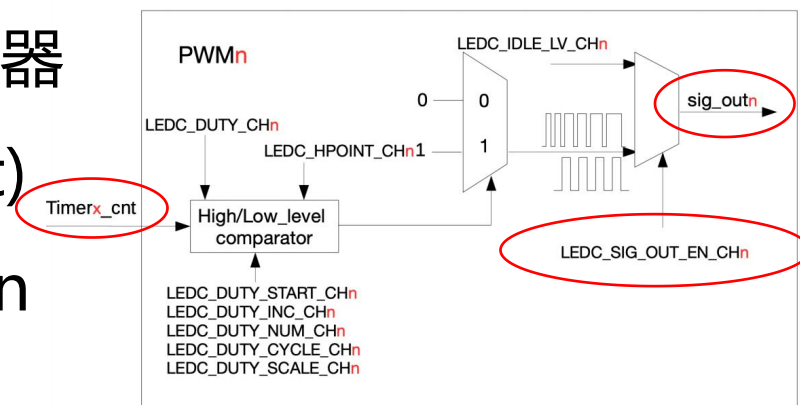
图 35.2-1. 定时器和 PWM 生成器功能块

定时器工作流程

以LEDC专用定时器产生LEDPWM为例

时钟源—预分频器—计数器—比较器

- 一个高低电平比较器+两个选择器
- 将定时器的计数值 (Timerx_cnt) 与高低电平比较器的值 Hpointn 和 Lpointn 比较。



- 如果 **Timerx_cnt == Hpointn**, 则 sig_outn 为 **1**。
- 如果 **Timerx_cnt == Lpointn**, 则 sig_outn 为 **0**。

定时器工作流程

以LEDC专用定时器产生LEDPWM为例

时钟源—预分频器—计数器—比较器

- 如果 **Timerx_cnt == Hpointn**, 则 sig_outn 为 **1**。
- 如果 **Timerx_cnt == Lpointn**, 则 sig_outn 为 **0**。

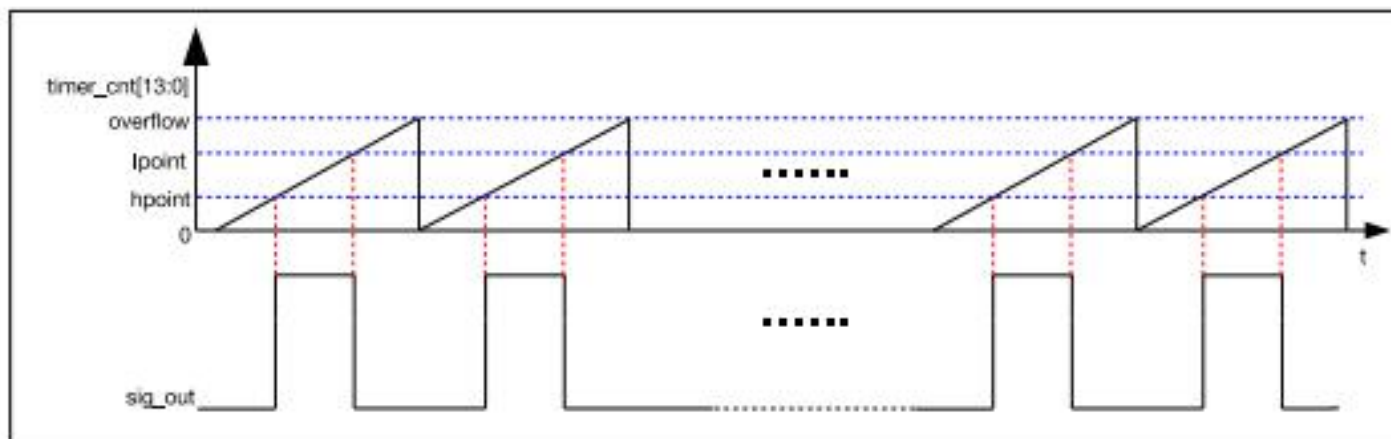


图 35.2-3. LED PWM 输出信号图

定时器特性：异步

- **硬件定时器和CPU异步工作**：定时器在自己的时钟驱动下不停计数，不受CPU执行影响，CPU亦然
- **优势**：CPU不用轮询检查效率高；定时由硬件保证不受CPU负载影响更为精准；支持低功耗（CPU休眠，定时器继续工作）
- **是否存在问题？**

定时器特性：异步

- **问题1：54位寄存器的读取问题**

- ESP32-S3的定时器组有54位的计数器，但CPU的数据总线是32位的。我们无法一次就读完54位的计数值，必须分两次读取：先读低32位，再读高22位（或者反过来）。
- 如果在**读低32位之后、读高22位之前**，计数器正好发生了**进位**（低32位从0xFFFFFFFF变成0x00000000，高22位加1），读取得到错误结果

- **对策：“锁存”机制**

- 向锁存寄存器写入任意值，触发锁存操作，硬件会把当前的54位计数值存到锁存寄存器中，然后我们分别读取锁存后的高32位和低32位，因为锁存的值是固定的，所以不会有中间进位的问题

定时器特性：异步

- **问题2：寄存器写入完成，不一定等于目标硬件已经在自己的时钟域采用了新值**
 - CPU/APB 配置时钟和定时器计数时钟可能不同；多位数值不能允许一半是旧值、一半是新值；分频值、报警值或重载值应在明确的边界整体生效；某些更新要避免在一个周期中途改变波形或产生毛刺。
- **对策：影子寄存器 (Shadow Register)**
 - 我们写入的配置值（如比较值、分频值）先存在“影子寄存器”中，影子寄存器的值不会立即生效，等到某个特定的时刻（如计数器溢出、下一个周期开始时），硬件才会自动把值加载到实际工作的寄存器中

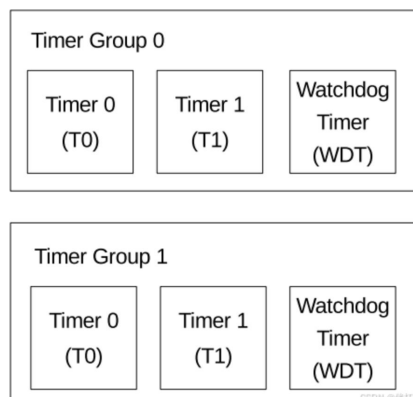
ESP32定时器资源

定时器类型	核心定位	典型使用场景
内核定时器	内核级高精度计时	FreeRTOS 任务调度节拍、代码性能耗时统计、纳秒级短延时
通用定时器	外设级通用定时 / 中断	周期性任务触发、输入捕获、脉冲计数、标准定时器中断教学
系统定时器	全局高精度系统时钟	esp_timer 软件定时器底层、微秒级高精度定时、操作系统时基
RTC 慢速定时器	低功耗定时唤醒	深度睡眠定时唤醒、低功耗下的长周期计时、RTC 时钟走时
主系统看门狗 (MWDT)	系统运行监控	防止程序死机跑飞, 超时触发中断或系统复位
RTC 看门狗	低功耗模式监控	启动流程监控、休眠状态异常复位
LEDC 定时器	PWM 时钟基准	LED 调光、呼吸灯、简易无源蜂鸣器驱动
MCPWM 定时器	电机控制 PWM 基准	直流电机调速、步进电机驱动、开关电源、伺服电机控制
RMT 通道计时单元	协议时序生成	红外遥控、WS2812 灯带、单总线协议 (如 DS18B20)

ESP32定时器资源

定时器组 (TIMG) - 通用定时器

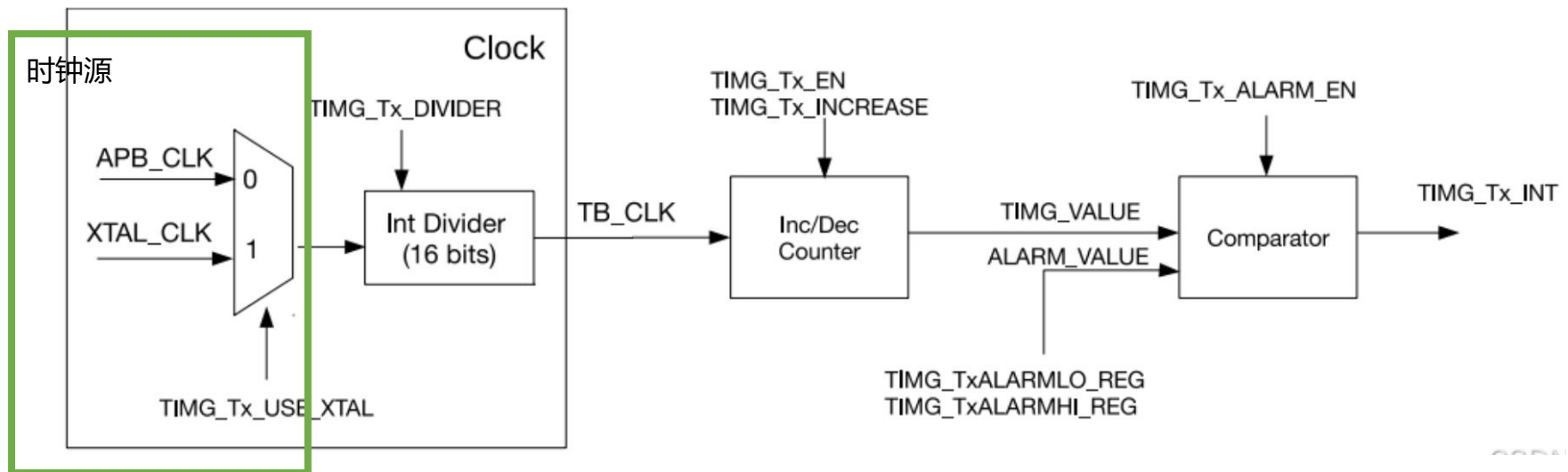
- ESP32 内置了 **2 组通用定时器**，每组包含 2 个 64 位的通用定时器和一个主系统看门狗定时器。
- 通用定时器用于准确设定时间间隔、在一定间隔后触发（周期或非周期的）中断或充当硬件时钟



ESP32定时器资源

定时器组 (TIMG) - 通用定时器

- 每个通用定时器都包括一个时钟选择器，一个 16 位的**时钟预分频器**，一个 54 位的**时基计数器**，一个比较器



ESP32定时器资源

定时器组 (TIMG) - 通用定时器

报警中断

定时器中断：溢出中断（周期中断）、比较器中断（报警中断）

- 定时器配置为在**当前值与报警值相同时触发报警**。报警会产生**中断**，（可选择）让定时器的当前值自动重新加载，电平中断在报警后触发。报警后，电平中断会一直被拉高，直至手动清除中断。
- 单次报警** & **周期性报警**

4.3. ESP32-S3的中断机制



ESP32定时器资源

看门狗定时器 (WatchdogTimer)

- 专门用于“**监控系统健康状态**”的特殊定时器
- 系统**正常**运行的时候会**定期**“喂狗”（**清零**看门狗计时器），一旦程序跑飞、陷入死循环或者出现死机等**异常**，“没人喂狗”，看门狗计数器会计数到**溢出**，然后**自动触发系统复位**，让设备重新启动恢复正常

ESP32-S3 中有三个数字看门狗定时器：两个定时器组（TIMG）中各有一个（称作主系统看门狗定时器，缩写为 MWDT），RTC 模块中有一个（称作 RTC 看门狗定时器，缩写为 RWDT）

ESP32定时器资源

系统定时器 (SYSTIMER)

- ESP32-S3芯片内置**一组52位系统定时器**，用于生成操作系统所需的**滴答定时中断**，也可用作普通定时器**生成周期或单次延时中断**
- 系统定时器内置**两个计数器** (UNIT0 和 UNIT1) 以及**三个比较器** (COMP 0、COMP1 和 COMP2)。比较器用于监控计数器的计数值是否达到报警值

ESP32定时器资源

系统定时器 (SYSTIMER) - 报警中断

p.s. ESP32-S3的系统定时器为52位的计数器，但CPU的数据总线是32位的。我们无法一次就读完52位的计数值，必须分两次读取：先读低32位，再读高20位

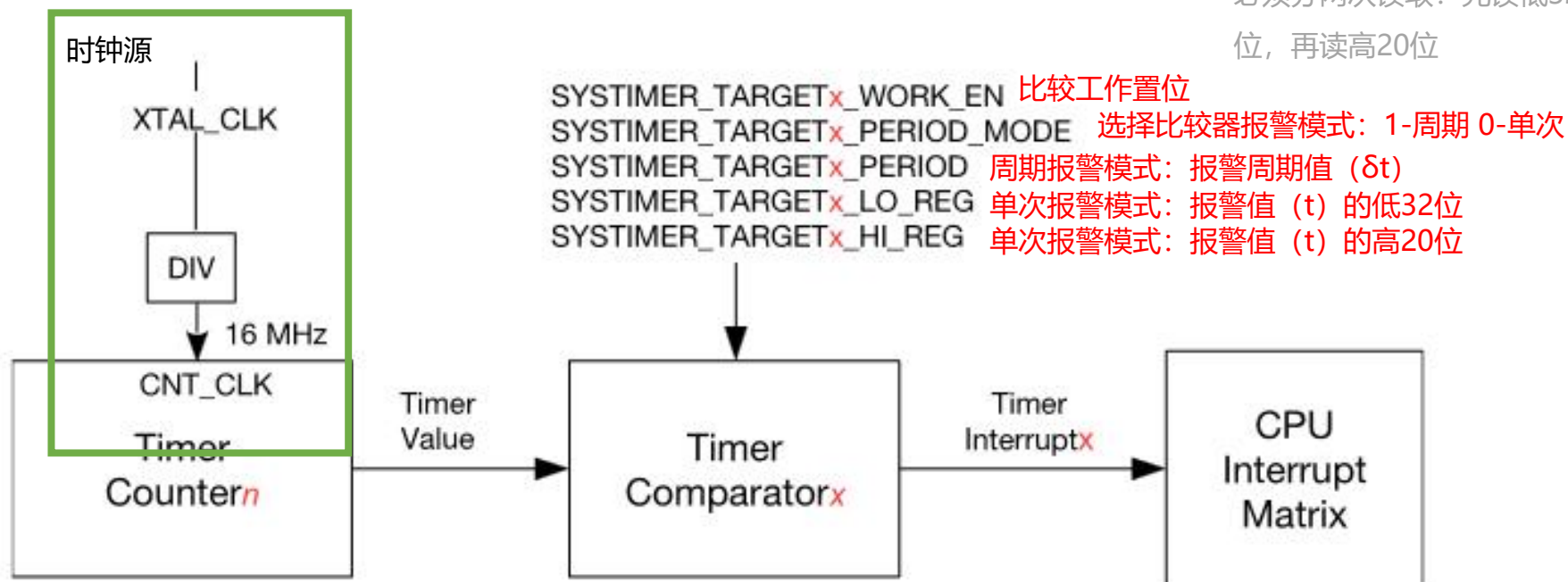


图 11.4-1. 系统定时器生成报警

ESP32定时器资源

系统定时器 (SYSTIMER) - 报警中断

- **周期**报警模式：假设当前计数值为 t_1 ，经过一段时间，当计数值达到 $t_1 + n\delta t$ 时，将触发中断，实现周期性报警
- **单次**报警模式：假设当前计数值为 t_2 ($t_2 \leq t$)，经过一段时间，当计数到报警值 (t) 时，则触发一次报警中断（仅一次）

表 11.4-2. 报警触发条件

t_c 与 t_t 的关系	触发条件
$t_c - t_t \leq 0$	当 $t_c = t_t$ 时，触发报警
$0 \leq t_c - t_t < 2^{51} - 1$ $(t_c < 2^{51} \text{ 且 } t_t < 2^{51};$ 或 $t_c \geq 2^{51} \text{ 且 } t_t \geq 2^{51})$	立即触发报警
$t_c - t_t \geq 2^{51} - 1$	t_c 达到最大值 52'hfffffffff 后溢出，然后从 0 开始计数，计数达到 t_t 时触发报警

ESP32的定时器资源

定时器本体与输出通道

- **本体**（时基单元）：时钟选择+分频器+计数器
- **通道**（输出单元）：比较器，输出引脚
- 本体决定频率，通道阈值决定占空比，**一个通道**同一时间只能以**一个本体**为时钟基准，**一个本体**可以连**多个通道**

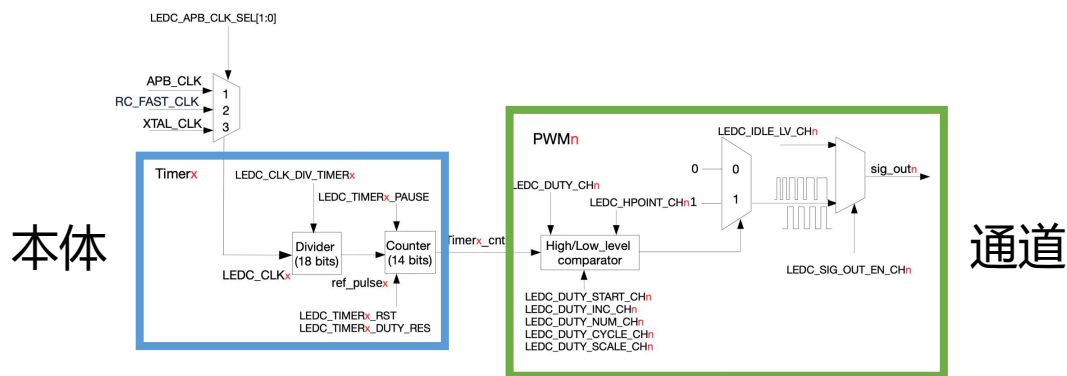


图 35.2-1. 定时器和 PWM 生成器功能块

ESP32的定时器资源

系统定时器 (SYSTIMER)

- 两个本体——三个通道

配置寄存器SYSTIMER_TARGETx_TIMER_UNIT_SEL
选择用于与COMPx进行比较的计数器值：
1-UNIT1 0-UNIT0

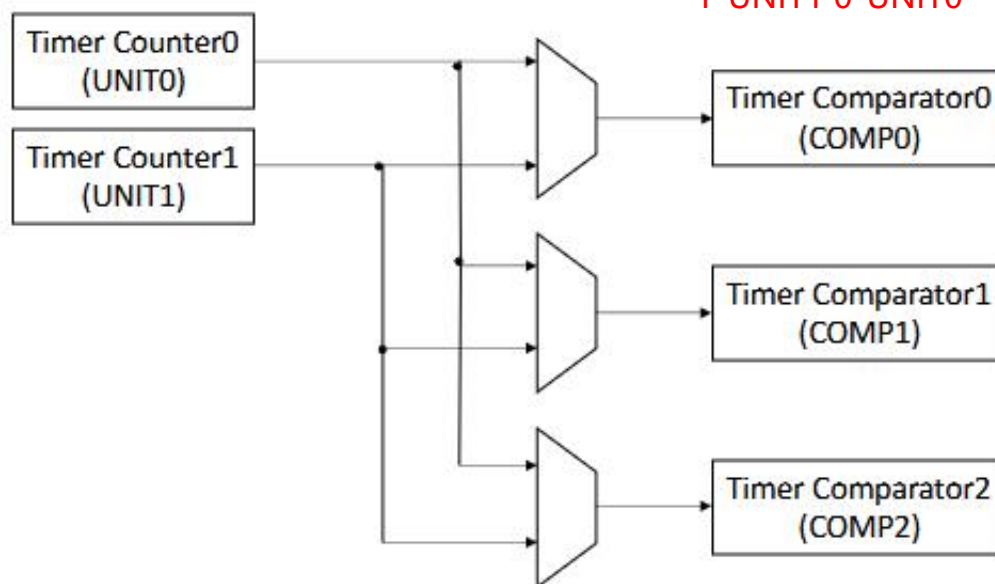


图 11.1-1. 系统定时器结构图

ESP32的定时器资源

LEDC专用定时器

- 有四个独立定时器，可以实现小数分频
- 四个本体——八个通道

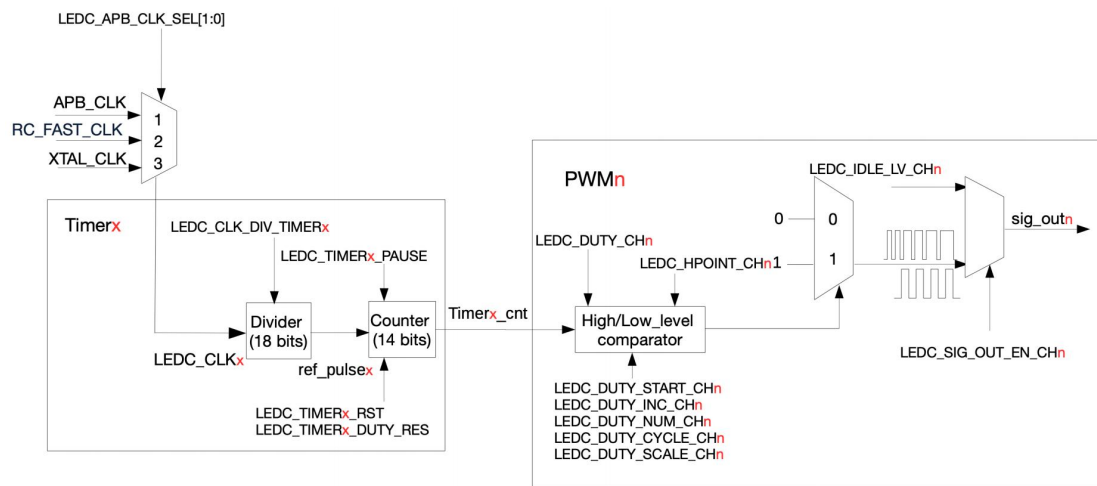


图 35.2-1. 定时器和 PWM 生成器功能块

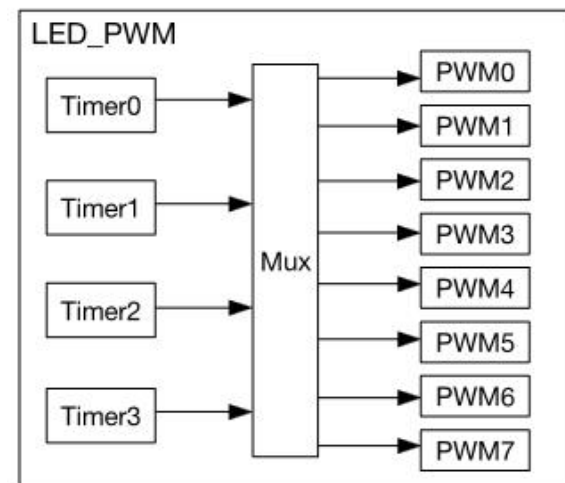


图 35.1-1. LED PWM 控制器架构

ESP32的定时器资源

MCPWM（电机控制脉宽调制器）专用定时器

- ESP32-S3中有 2 个MCPWM外设，每个里面有 3 个 PWM定时器，3个PWM操作器，每个操作器对应 2 路 PWM输出

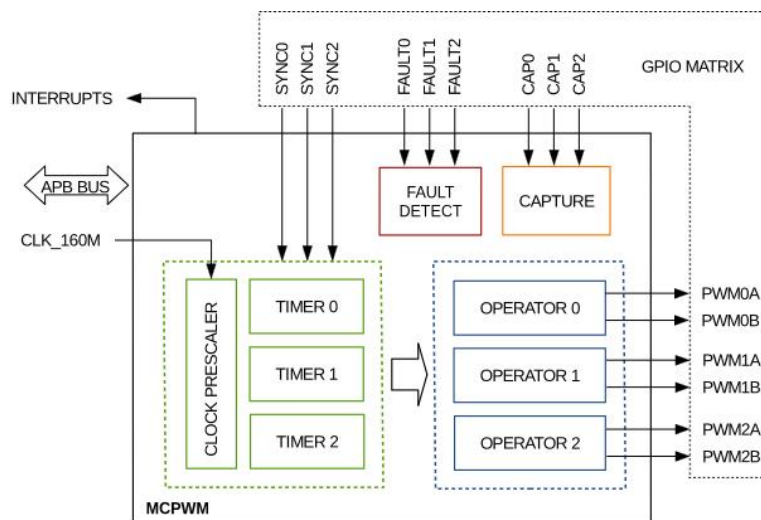


图 36.2-1. MCPWM 外设概览

ESP32的定时器资源

MCPWM专用定时器

- 每个**操作器**都能选择使用**任意一个**PWM**定时器**作为其时间基准。
- 多个操作器可以通过使用 GPIO 交换矩阵的同步信号，内置**同步逻辑**允许一个或多个 MCPWM 外设中的多个 PWM 定时器模块作为一个系统协同工作，产生**同步的 PWM 输出**（例如多路输出保持相同频率和相位），也可以产生彼此独立的PWM波形。

ESP32的定时器资源

MCPWM专用定时器 – 工作模式

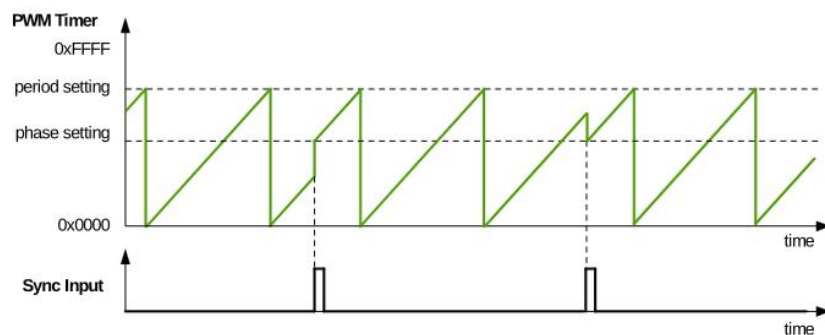


图 36.3-6. 递增计数模式波形

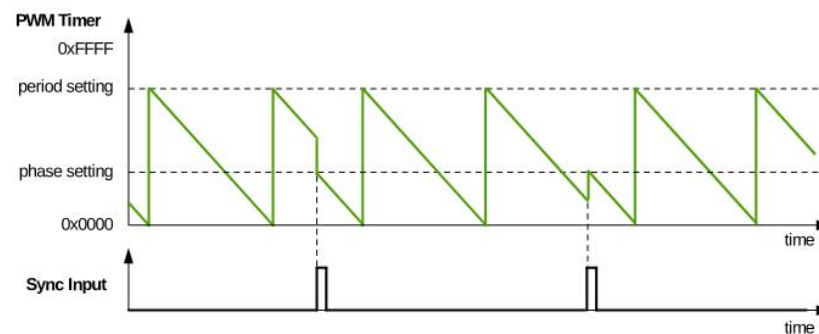


图 36.3-7. 递减计数模式波形

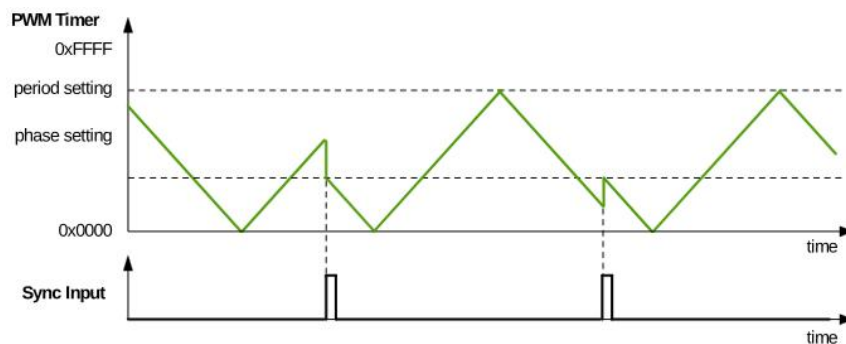


图 36.3-8. 递增递减循环模式波形，同步事件后递减

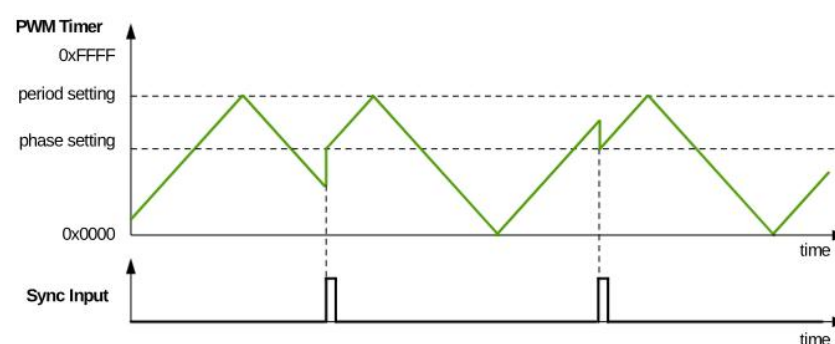


图 36.3-9. 递增递减循环模式波形，同步事件后递增

定时器资源使用

• 通用定时器

函数原型	核心功能	参数说明
<code>hw_timer_t* timerBegin(uint32_t frequency)</code>	分配空闲通用定时器, 配置计数频率	frequency: 计数器计数频率 (单Hz)
<code>void timerAttachInterrupt(hw_time r_t *timer, void (*userFunc)(void))</code>	绑定定时器中断回调 函数, 计数值匹配报 警值时触发执行	timer: 定时器对象指针; userFunc: 中断服务函数, 需加 IRAM_ATTR 修饰保证响应速度
<code>void timerAlarm(hw_timer_t *timer, uint64_t alarm_value, bool autoreload, uint64_t reload_count)</code>	设置报警阈值, 配置 自动重载模式	alarm_value: 触发中断的计数值; autoreload: true = 周期模式, false = 单次模式 reload_count: 重载次数, 0 表示无限次

上述函数声明基于ArduinoIDE版本 2.3.10 (其他前后版本可能函数声明不太一样, 但大体用法和意思是一样的)

定时器资源使用

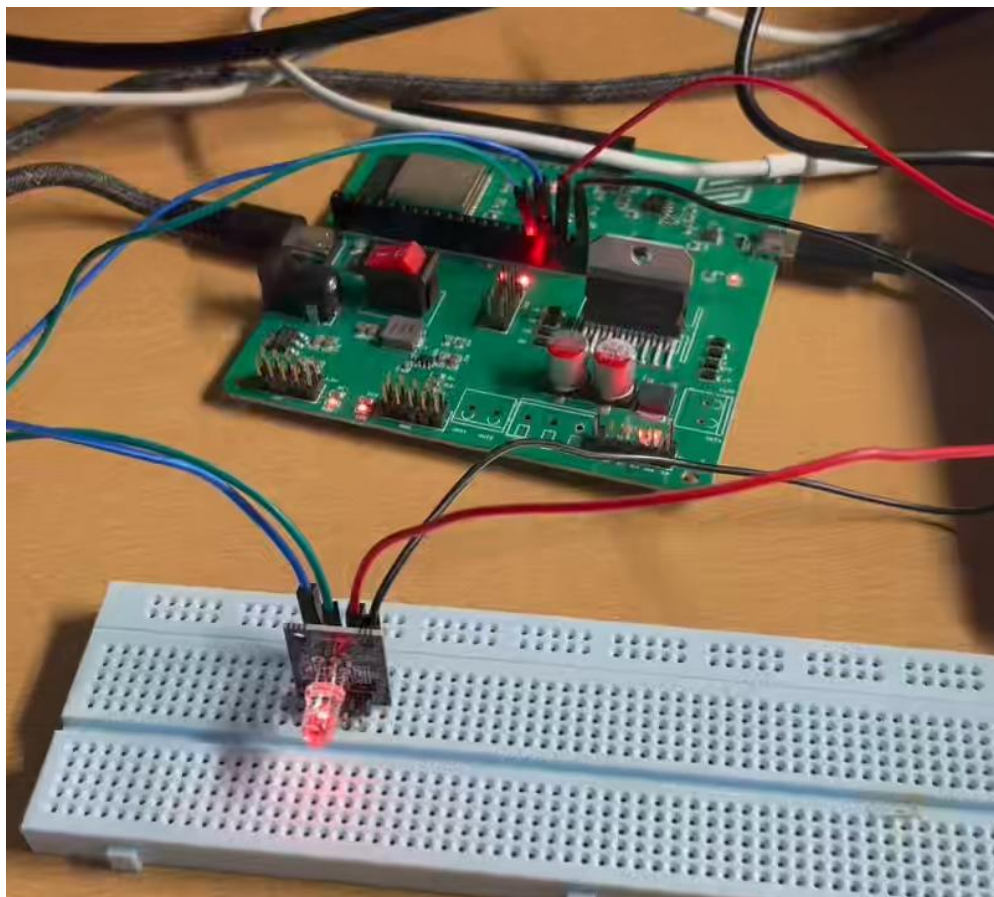
- LEDC 专用定时器：一个定时器可带多路同频通道；一路通道同一时间只能绑定一个定时器**

函数原型	核心功能	参数说明
<pre>bool ledcAttachChannel(uint8_t pin, uint8_t channel, uint32_t freq, uint8_t resolution);</pre>	将物理 GPIO 引脚与 LEDC 通道绑定输出 PWM	pin: 输出 PWM 的物理 GPIO 引脚号 channel: LEDC 输出通道编号 freq: PWM 输出频率 resolution: PWM 占空比分辨率
<pre>void ledcDetach(uint8_t pin)</pre>	解除引脚与 LEDC 通道的绑定，恢复普通 IO 功能	pin: 输出 PWM 的物理 GPIO 引脚号

上述函数声明基于ArduinoIDE版本 2.3.10（其他前后版本可能函数声明不太一样，但大体用法和意思是一样的）

定时器资源使用

实验 1：定时器中断—LED闪烁



定时器资源管理

- **必要性：**通用专用定时器硬件资源**数量有限**，若多功能同时占用同一定时器硬件会导致计数紊乱、定时不准、PWM波形畸变、中断异常等等
- **原则：****专用优先、同频复用、按需释放**、系统资源避让
(Arduino的delay()等系统函数基于SYSTIMER，不占用TIMG)

定时器资源管理

- 案例需求：**编码器采集：每 5ms (200Hz) 触发一次中断，读取电机脉冲数（定时任务）；左右直流电机调速：PWM 频率 20kHz，占空比独立可调；蜂鸣器报警：输出 2.5kHz 方波，实现报警提示；状态呼吸灯：PWM 频率 1kHz，占空比渐变实现呼吸效果

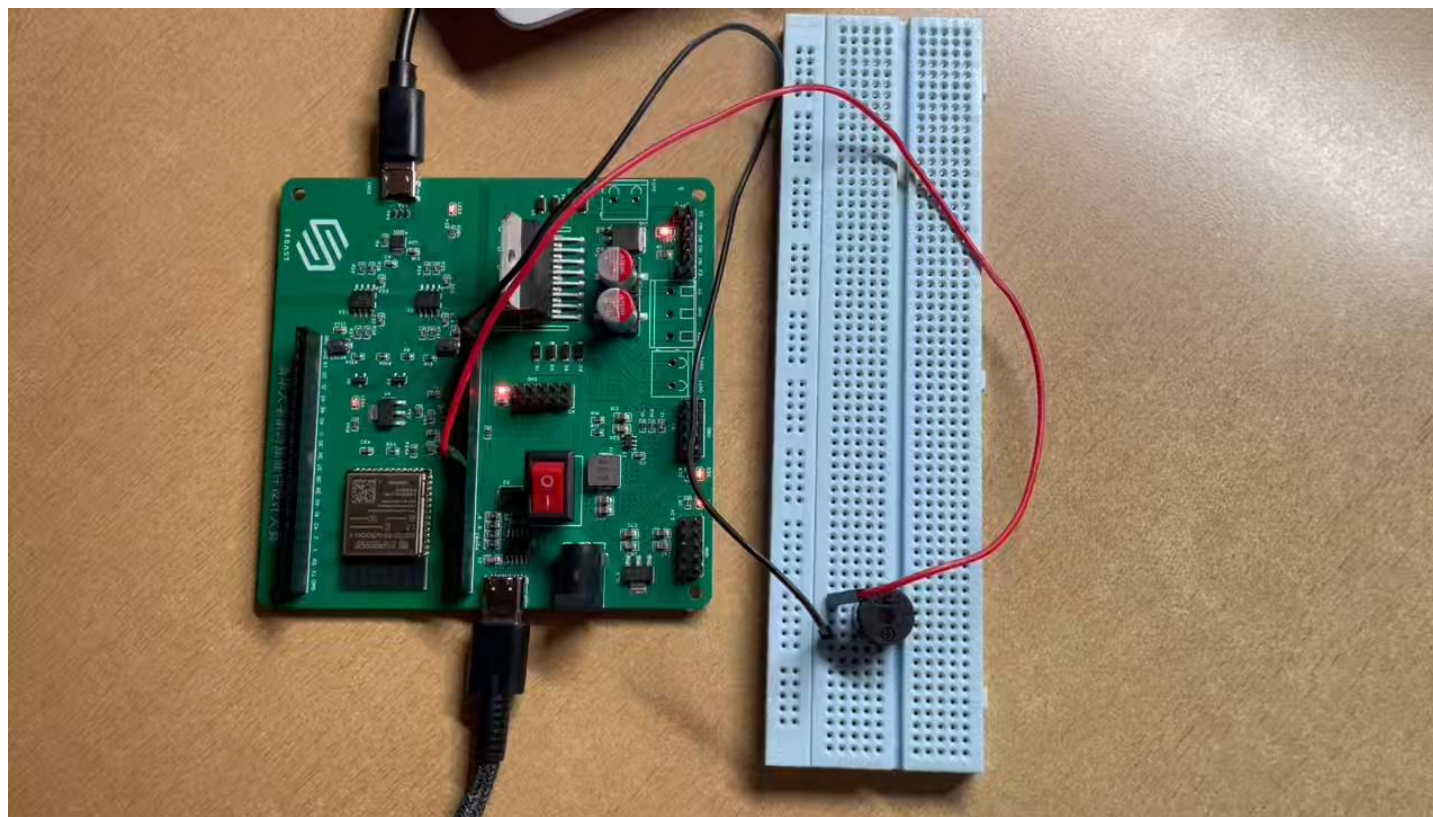
功能需求	工作频率	分配硬件资源	对应通道 / 编号
编码器定时采集	200Hz	通用定时器 0	——
左 / 右电机调速（2 路）	20kHz	LEDC Timer0	通道 0、通道 1
蜂鸣器报警	2.5kHz	LEDC Timer1	通道 2
状态呼吸灯	1kHz	LEDC Timer2	通道 4

定时器应用：无源蜂鸣器

- 有源蜂鸣器：内部已有振荡器，GPIO 给高/低电平即可响/停。
- **无源**蜂鸣器：受迫振动（电信号→相应频率的机械振动→发声），**需要外部周期方波**
 - 方波**频率**决定**音高**

定时器应用：无源蜂鸣器

实验2：定时器中断——驱动蜂鸣器



定时器应用：无源蜂鸣器

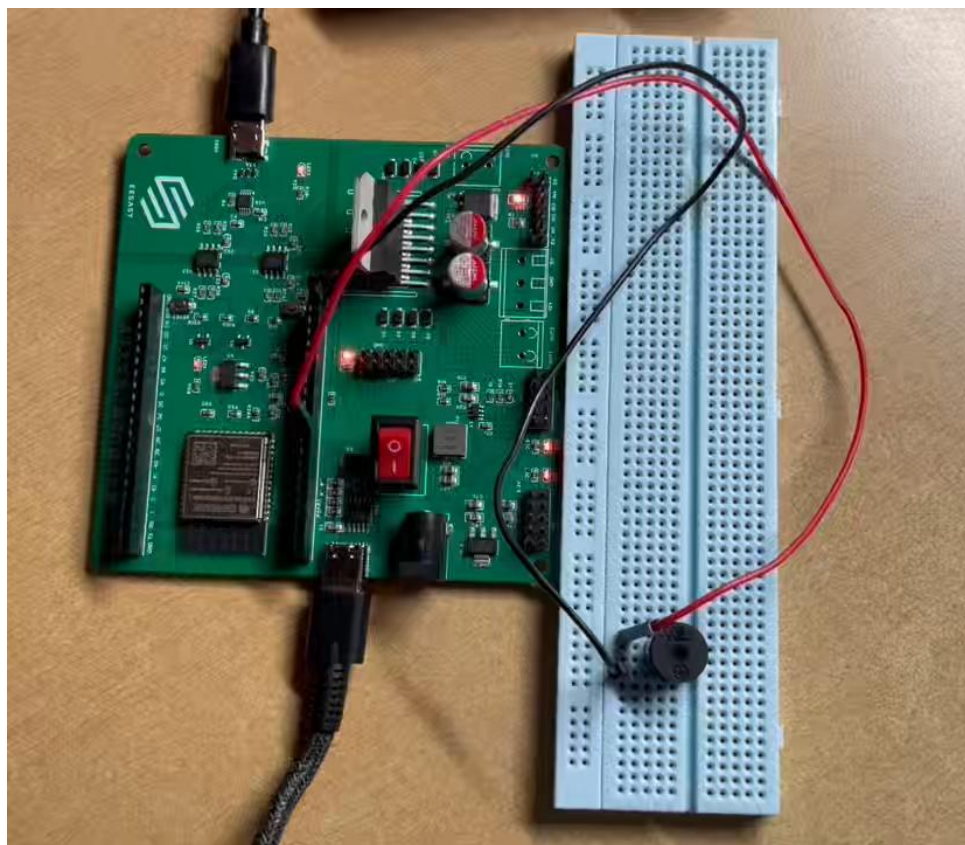
- Arduino提供**封装好的tone函数**，可直接在对应引脚输出某频率的方波，持续一定时间。
- 频率单位：Hz
- 持续时间单位：ms

音符	频率/Hz	音符	频率/Hz	音符	频率/Hz
低音1	262	中音1	523	高音1	1046
低音1#	277	中音1#	554	高音1#	1109
低音2	294	中音2	587	高音2	1175
低音2#	311	中音2#	622	高音2#	1245
低音3	330	中音3	659	高音3	1318
低音4	349	中音4	698	高音4	1397
低音4#	370	中音4#	740	高音4#	1480
低音5	392	中音5	784	高音5	1568
低音5#	415	中音5#	831	高音5#	1661
低音6	440	中音6	880	高音6	1760
低音6#	466	中音6#	932	高音6#	1865
低音7	494	中音7	988	高音7	1976

```
void tone(uint8_t _pin, unsigned int frequency,  
          unsigned long duration)
```


定时器应用：无源蜂鸣器

实验3：使用tone函数让蜂鸣器播放音乐

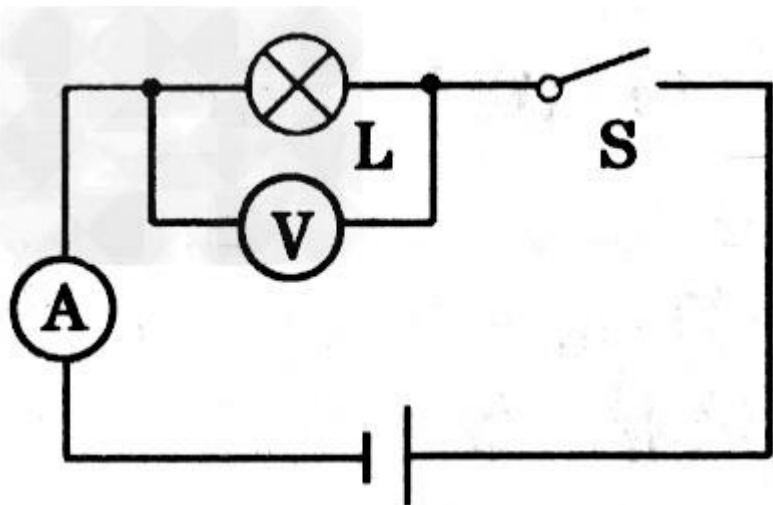


02 PWM 简介与原理

以下内容基于 ESP32-S3 (LEDC + MCPWM)



引入



1. 传统变阻器调光



低效率

2. PWM式开关控制



高效率，无热损耗
平均功率控制 = PWM



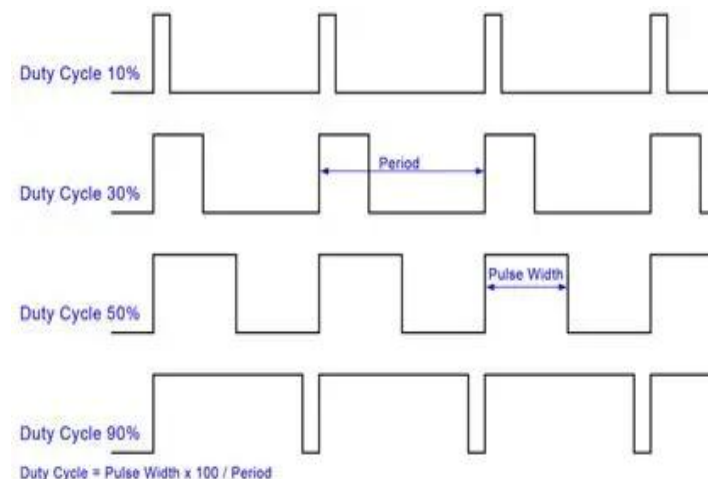
PWM（脉冲宽度调制）基本概念

核心定义

PWM 是一种用数字信号模拟“中间状态”的技术，方波的占空比可调，通过不同的占空比来传递不同的信息/能量。

PWM 的本质

PWM 不是独立外设，而是定时器加了一个比较门闸后的副产品



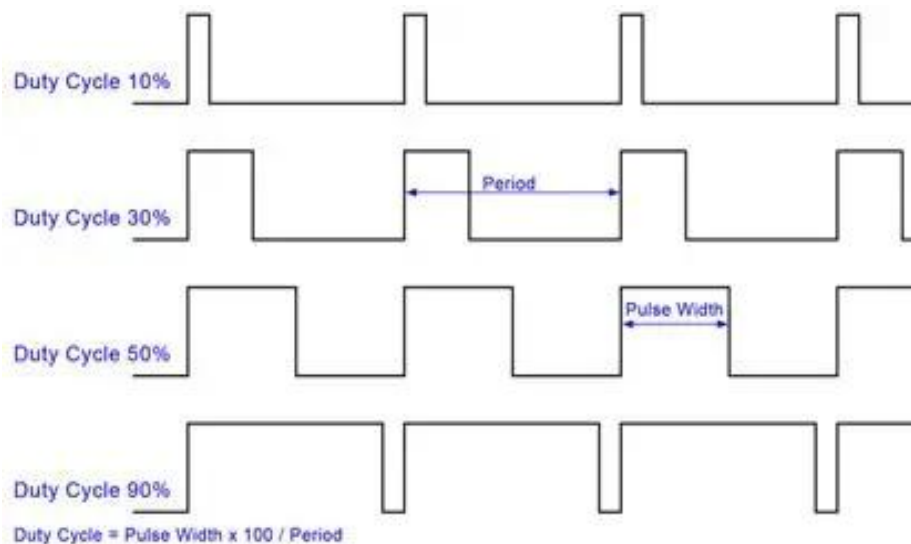
PWM（脉冲宽度调制）基本概念

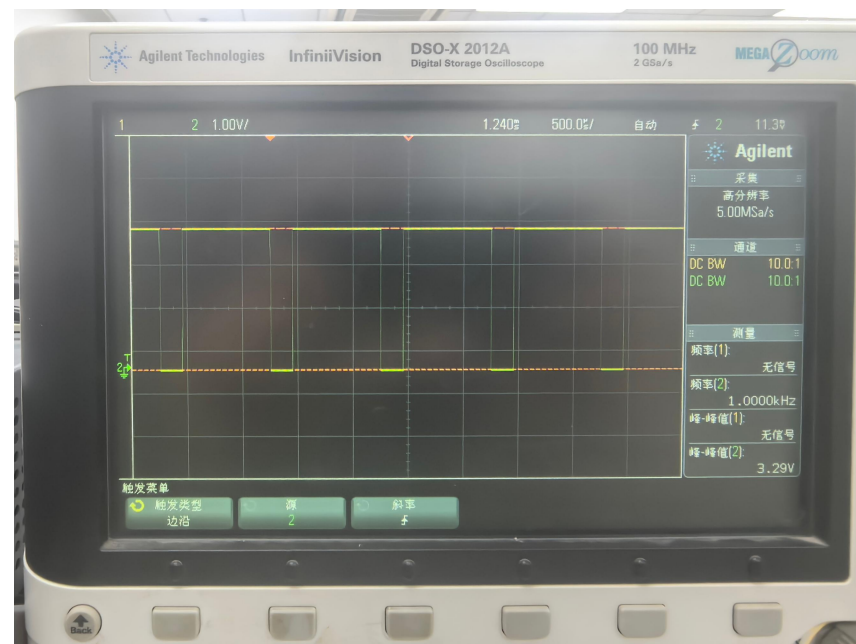
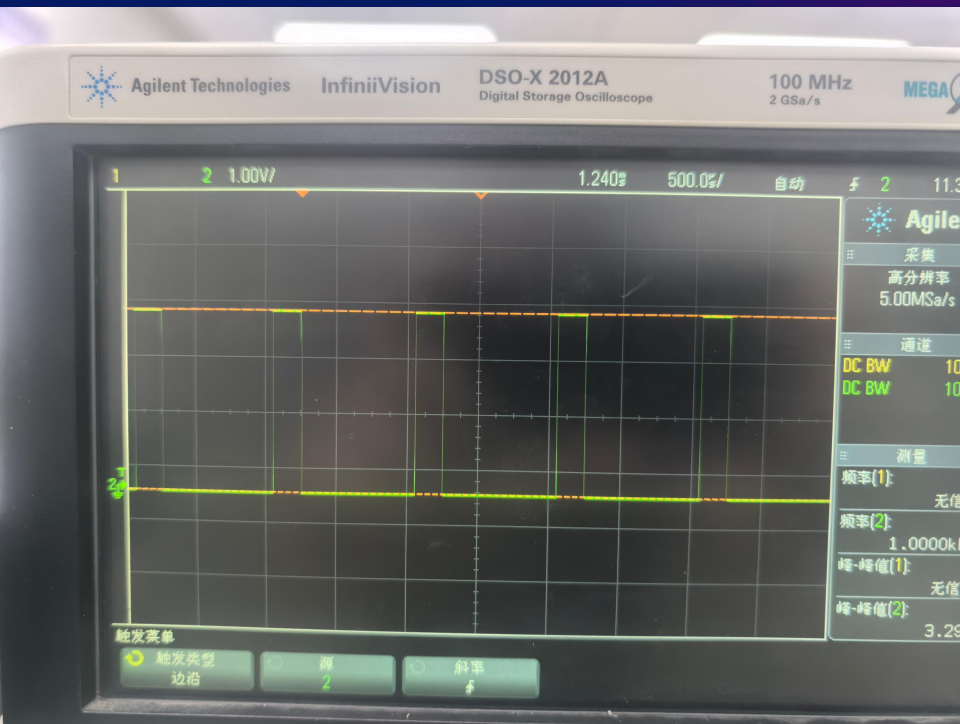
两个关键参数

周期 T / 频率 f —— 决定 PWM 信号重复的快慢

占空比 (Duty Cycle) —— 决定一个周期内高电平的比例

占空比 = (高电平时间 / 周期) $\times 100\%$





PWM 生成机制 —— 以 ESP32-S3 LEDC 为例

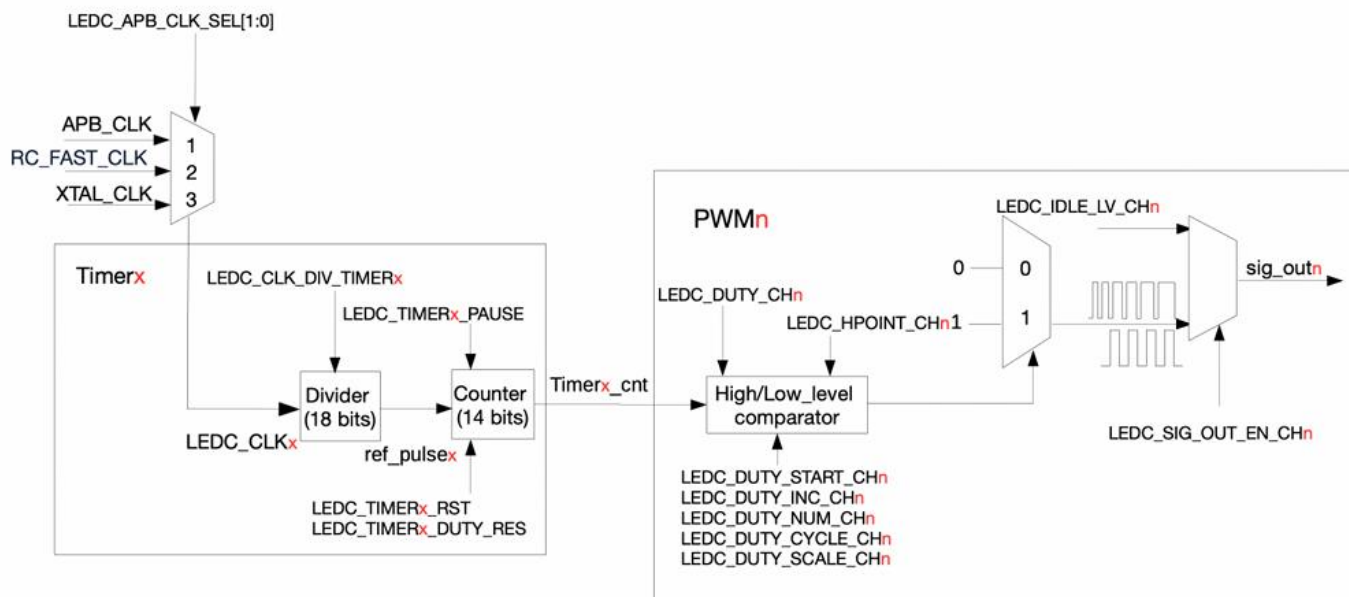


图 35.2-1. 定时器和 PWM 生成器功能块

LEDC 定时器决定 PWM 周期

时钟源 LEDC_CLKx

→ 进入时钟分频器

→ 产生 ref_pulsex(分频后的时钟)

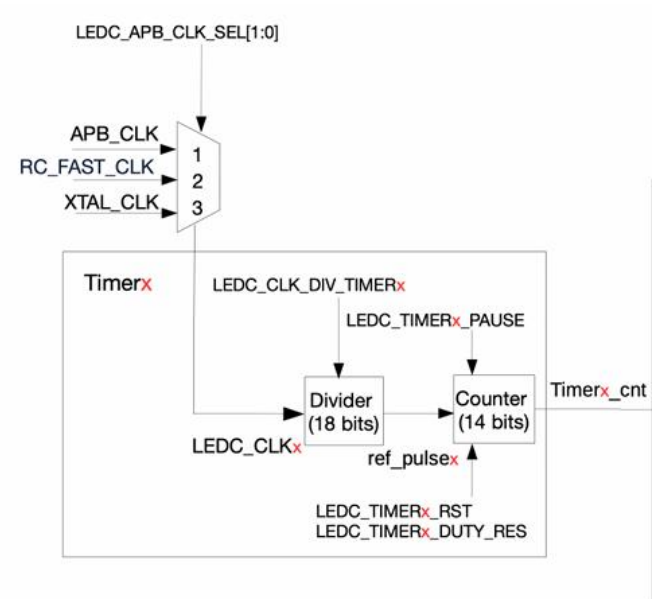
→ 驱动 14 位计数器递增

最大值 = $2^{\text{DUTY_RES}} - 1$

到达最大值 → 溢出归零 → 开始下一轮

PWM 频率公式:

$$f_{\text{PWM}} = \frac{f_{\text{LEDC_CLK}} (\text{时钟源频率})}{(\text{LEDC_CLK_DIV} (\text{分频系数}) \times 2^{\text{DUTY_RES}} (\text{计数周期}))}$$



PWM 生成器决定占空比

每个通道用 HPoint / LPoint 机制：

$CNT == HPoint \rightarrow$ 输出变高

$CNT == LPoint \rightarrow$ 输出变低

$LPoint = HPoint + LEDC_DUTY_CHn$

占空比 = $(LPoint - HPoint) / 2^{DUTY_RES}$

HPoint 控制输出相位，DUTY 控制高电平宽度

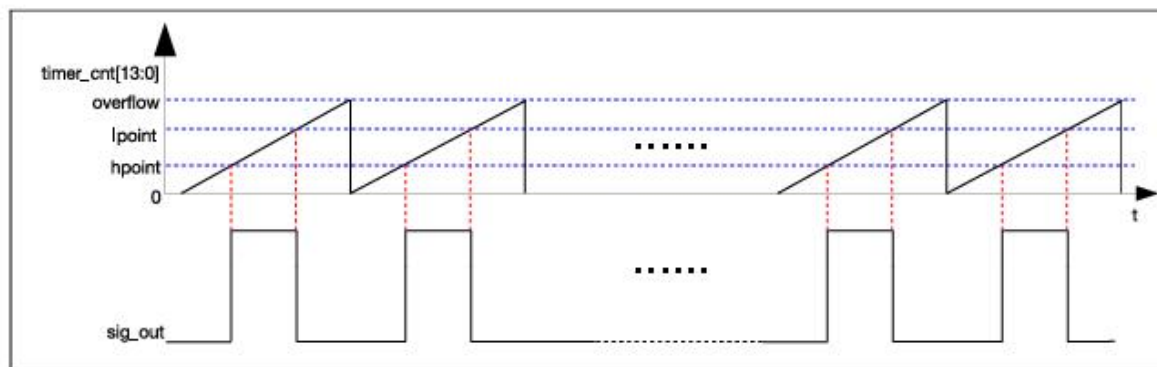
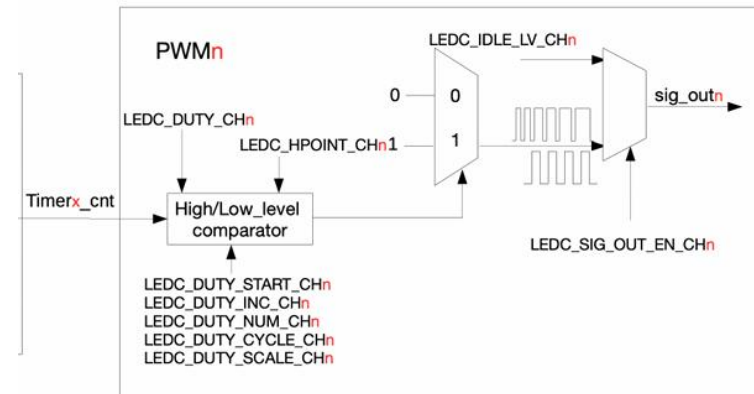


图 35.2-3. LED PWM 输出信号图

ESP32-S3 有两类 PWM 外设

LEDC (LED PWM 控制器) —— 适合大多数场景

MCPWM (电机控制 PWM) —— 专为电机设计

注：具体机理参考网络学堂中颁发的有关 ESP32 的数据手册

PWM资源

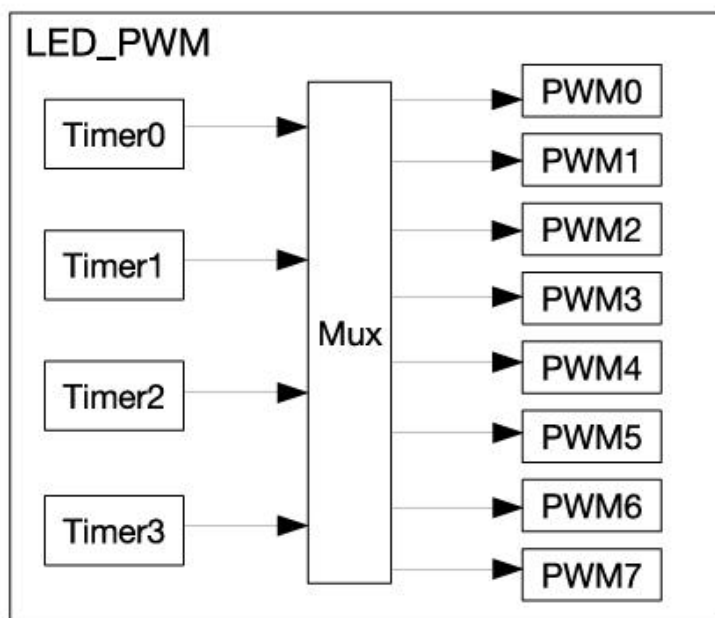


图 35.1-1. LED PWM 控制器架构

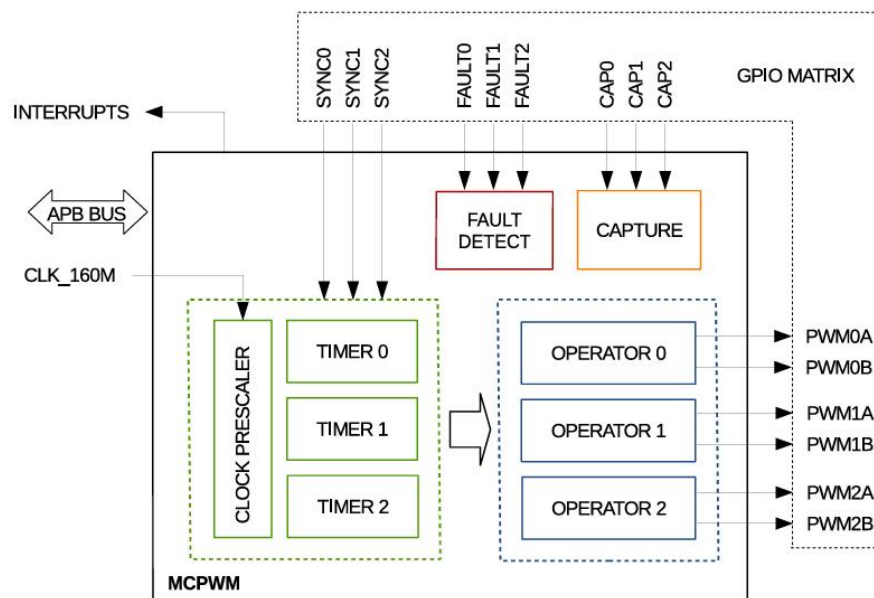


图 36.2-1. MCPWM 外设概览

PWM控制

同一个 PWM, “怎么取出信息”决定它积不积分



量这一个脉冲的宽度
→ 直接得一个值

不积分

逐脉冲测量 · 无平均
舵机、输入捕获



把很多周期抹平成均值
单看一个周期无意义

积分

必须平均才有值
电机、LED、加热器、DAC

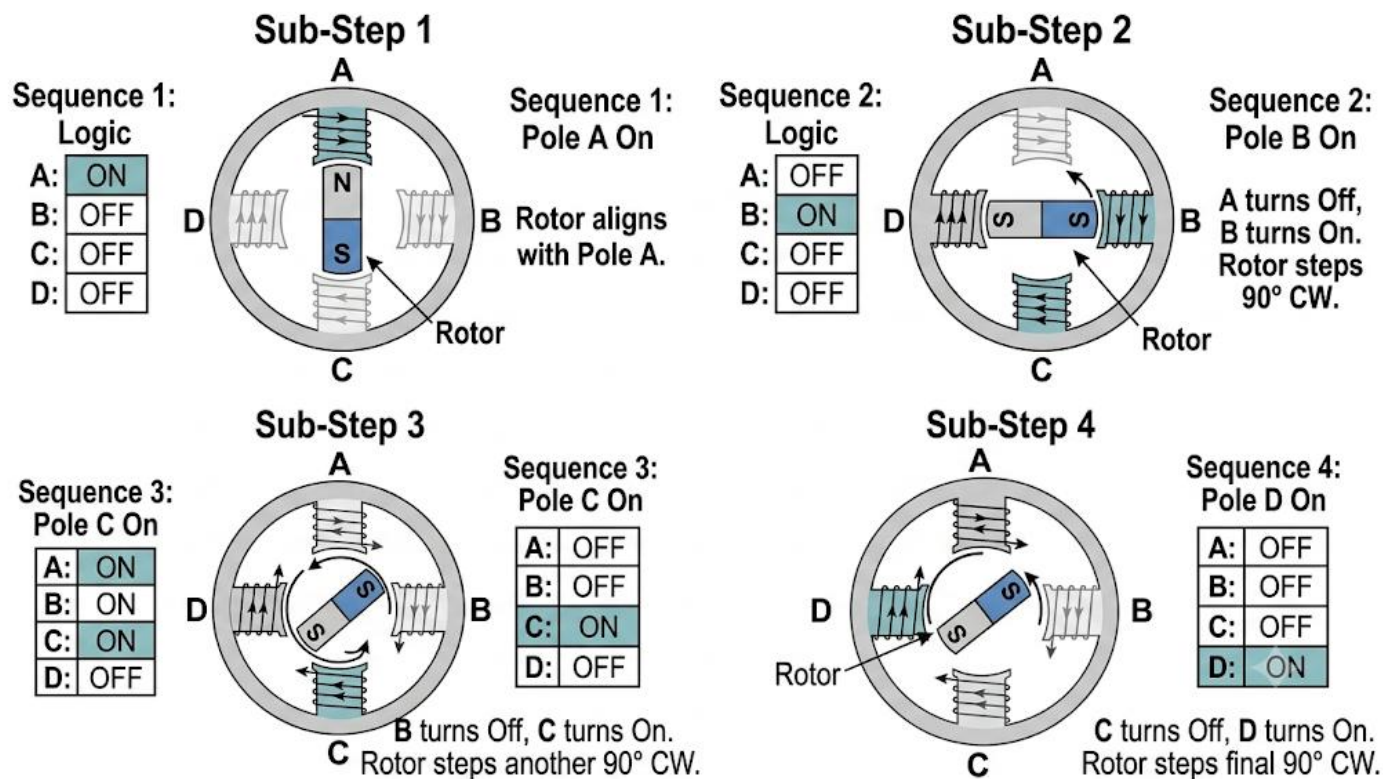


每个上升沿数一下
1、2、3、4 ...

不积分

逐周期计数 · 无平均
蜂鸣器膜片、步进、电荷泵

PWM信息-以四相步进电机为例



频率控旋转速度， 占空比控电机力量； 相位定旋转方向

阻塞式 delay vs 硬件 PWM 输出

LEDC/MCPWM 一旦配置完成，**PWM** 信号由硬件持续生成，**CPU** 完全解放出来做其他事！

阻塞式傻等 delay(1000)

类比：盯着水壶烧水 10 分钟，期间不许做任何其他事

```
while(1) {  
    LED_Toggle();  
    delay_ms(1000); // CPU空转  
    if(Button()){Do();} // 按键完全失效!  
}
```

CPU 在空循环中耗费功耗，对外界输入完全视若无睹

PWM 硬件自动生成

类比：带响铃的电水壶——烧上后该干什么干什么，水开了「叮」一声再处理

```
// 主循环零延迟  
while(1) {  
    if(Button()){Do();} // 实时响应  
}  
  
// PWM 由硬件自动输出，不需 CPU 持续干预
```


03 PWM应用-analogWrite



analogWrite() —— 最简单的 PWM 输出方式

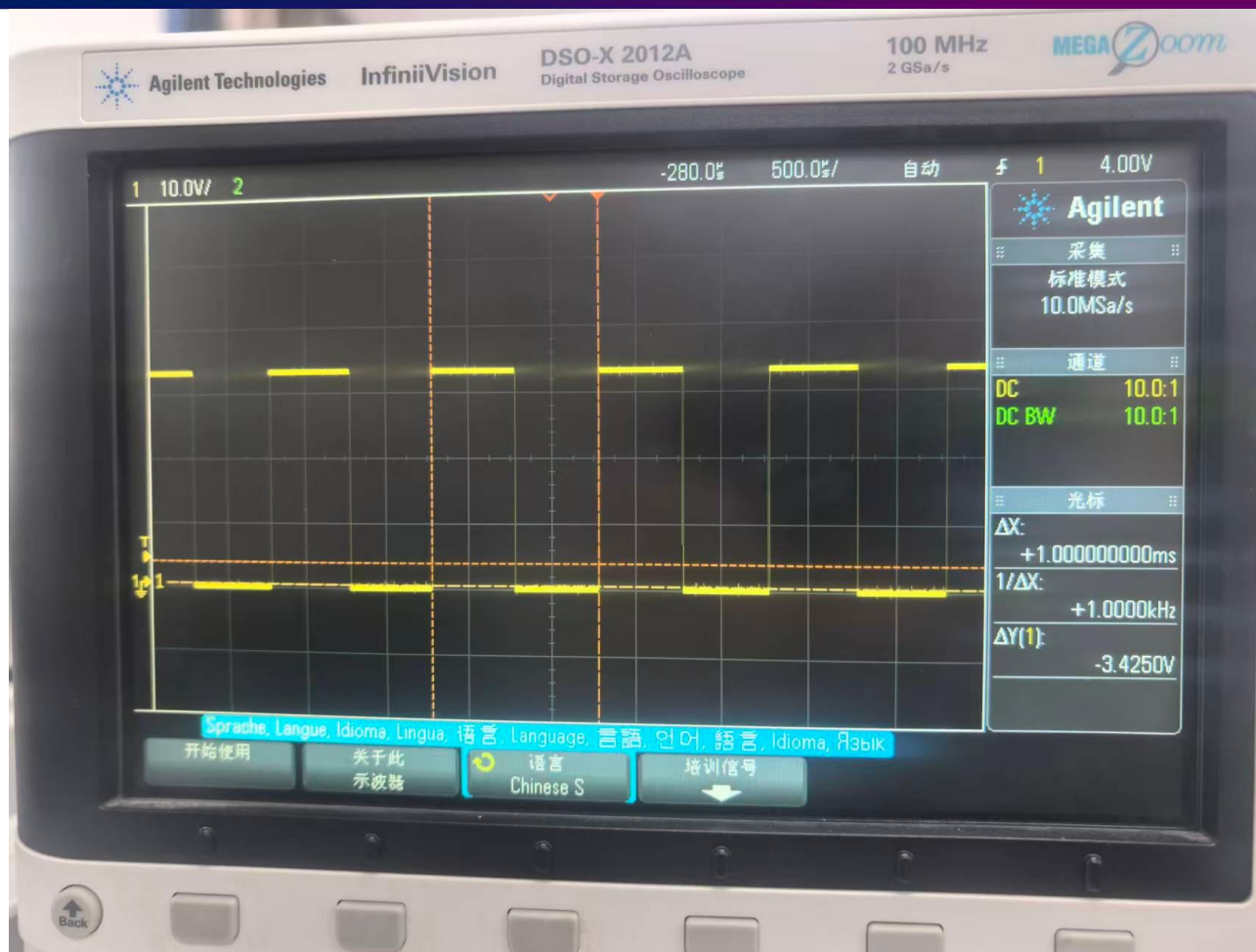
函数原型: **analogWrite(pin, value)**

value 取值 0~255, 对应占空比 0%~100%, 数值越大占空比越高

在 ESP32-S3 上, Arduino 核心会自动把 analogWrite() 映射到底层的 LEDC —— 自动分配定时器和通道, 不需要手动配置, 默认 PWM 频率约 1 kHz

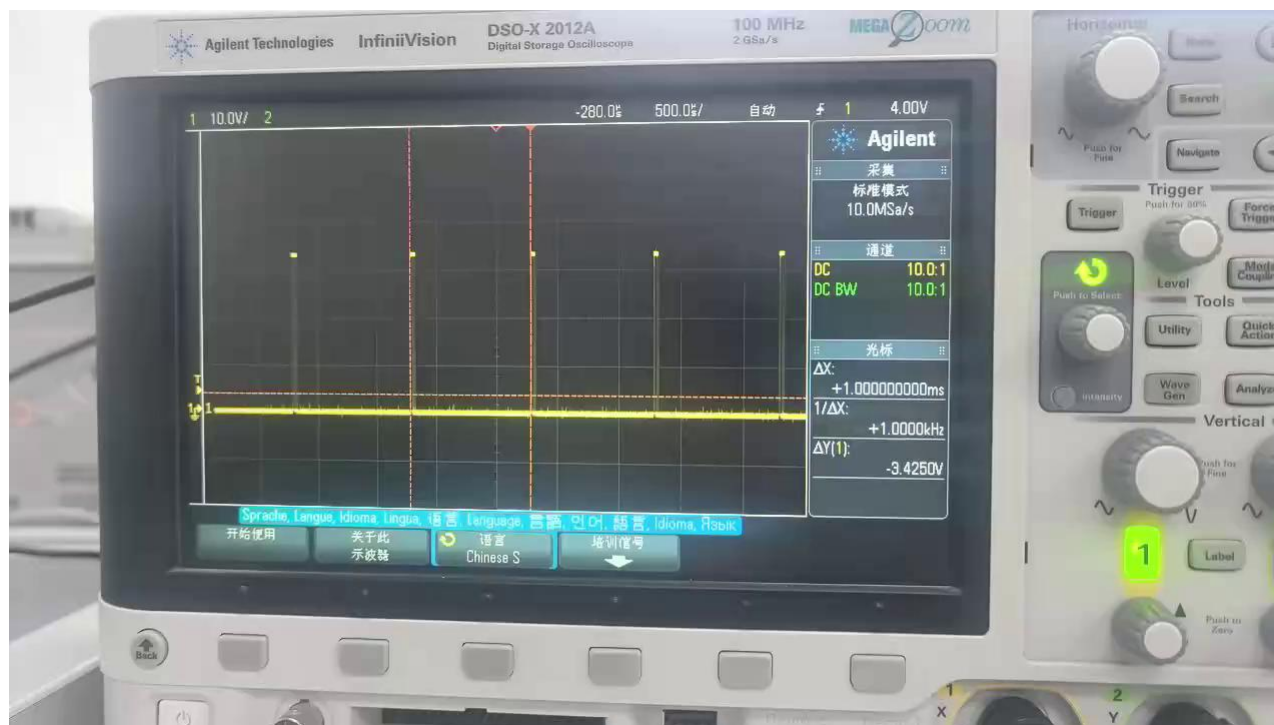
LED 呼吸灯 (占空比从 0 渐变到 255)

```
for (int i = 0; i <= 255; i++) {  
    analogWrite(ledPin, i);  
    delay(10);  
}
```



用电压表册是1.6V

渐变



ledcAttach() 常见踩坑

频率和分辨率不是随便配的：组合一旦超出LEDC硬件上限，ledcAttach() 会直接返回 **false**，且不报错、不中断程序

硬件限制：分频系数(LED_{CLK_DIV}) 上限约 1024，分辨率(DUTY_RES) 上限 14 位

判断公式：分频系数 = $f_LEDCLK / (\text{频率} \times 2^{\text{分辨率}})$

例：舵机 50Hz 配 16 位分辨率 → 超出14位上限，直接失败

例：直流电机 100Hz 配 8 位分辨率 → 算出分频系数约3125，远超1024上限，同样失败

排查建议：务必检查 ledcAttach() 的返回值，失败时打印提示

```
if (!ledcAttach(pin, freq, res)) {  
    Serial.println("ledcAttach失败！检查频率/分辨率组合");  
}
```

04 PWM 应用 —— 舵机

使用 PWM 控制舵机角度



舵机 (Servo) 控制协议

控制协议

标准频率：50 Hz，周期 = 20 ms

高电平脉宽决定角度：

高电平脉宽	角度
0.5 ms	0°（极左）
1.0 ms	45°
1.5 ms	90°（中位）
2.0 ms	135°
2.5 ms	180°（极右）

占空比范围：2.5% (0.5ms) ~ 12.5% (2.5ms)

连线注意

信号线 → GPIO (PWM 输出)；

电源线 → 5V（不是 3.3V！）

电流可达 **1A** 以上，多舵机时必须
用外部电源！

舵机小贴士：避免堵转

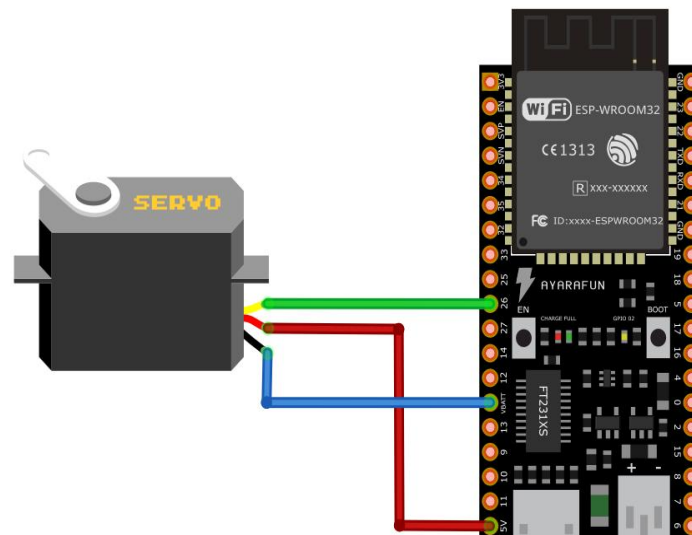
不同舵机（哪怕同型号不同批次）实际机械可转动范围不完全一样

打到理论上的 0° 或 180° 极限角度时，如果实际机械行程比这个小，会出现齿轮卡死、电机持续发出“嗡嗡”声（堵转）的情况

长时间堵转容易烧毁电机或磨损齿轮

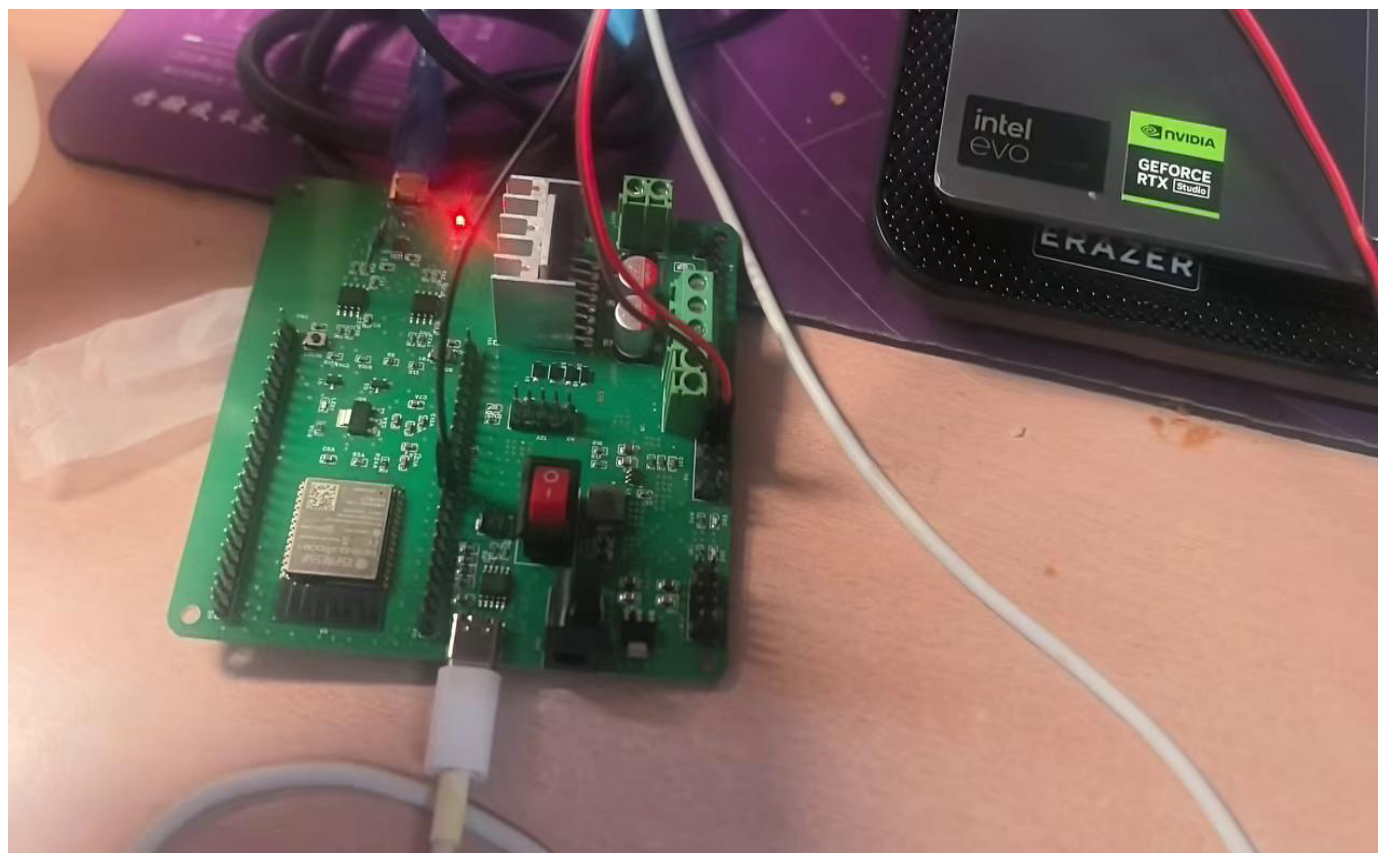
建议：第一次测试先用较窄的角度范围（比如 $60^\circ \sim 120^\circ$ ），确认没有异响后再逐步扩大到完整的 $0^\circ \sim 180^\circ$

案例：雨刷器效果



黄色/橙色线（信号线）：连接到 ESP32 的 **GPIO**。
红色线（电源正极）：连接到 ESP32 的 **5V** 引脚
棕色/黑色线（地线）：连接到 ESP32 的 **GND** 引脚

实验演示



05 电机控制



为什么不能直接用 GPIO 驱动电机？

两个致命问题

问题一：电流不足

GPIO 引脚输出能力仅 10~20 mA

电机启动需求 几百 mA ~ 几 A

单片机根本带不动！

问题二：芯片烧毁（反向电动势）

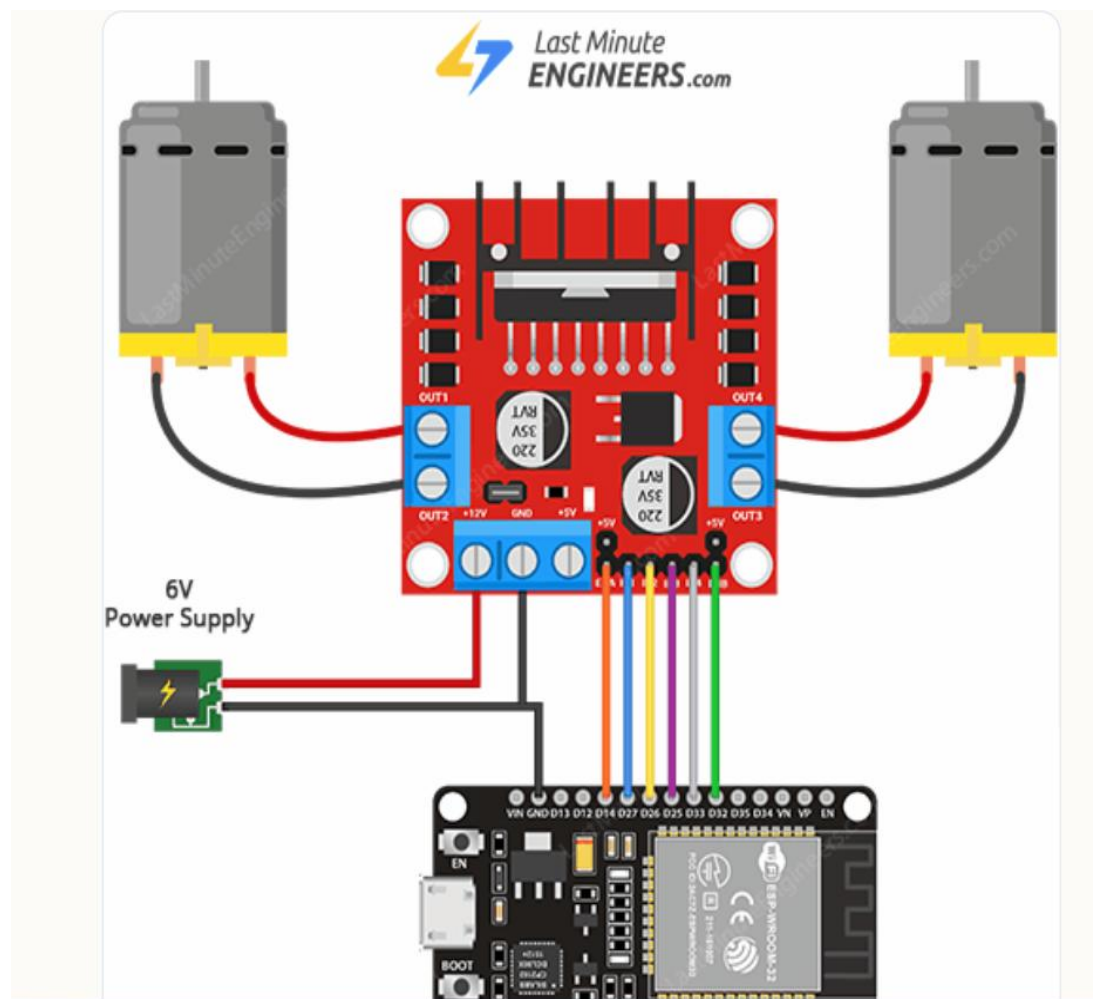
电机是感性负载（内部是线圈）

电机停止瞬间会产生高压反向电动势

电压直接灌回 GPIO 引脚，瞬间烧毁 MOS 管！

最致命的是可能会烧电脑！！

电机控制接线逻辑 (以 L298N 为例)



接线表

信号/引脚	一端（来源）	另一端（去向）	说明
IN1	单片机 GPIO	驱动板 IN1	方向控制位 1
IN2	单片机 GPIO	驱动板 IN2	方向控制位 2
ENA	单片机 PWM 引脚	驱动板 ENA	速度控制（PWM 占空比）
GND	单片机 GND	驱动板 GND	共地，必须接
OUT1	驱动板 OUT1	电机端子 A	电机一根线
OUT2	驱动板 OUT2	电机端子 B	电机另一根线
VS/+12V	电机电源正极	驱动板电源输入	给电机供电
VSS/+5V	5V 电源	驱动板逻辑供电	给芯片逻辑供电
GND	电源负极	驱动板 GND	电源地与共地相连

IN1	IN2	ENA (占空比)	电机状态
1	0	>0	正转，越高越快
0	1	>0	反转，越高越快
1	0	0	停止
0	1	0	停止
1	1	任意	停止（刹车）
0	0	任意	停止（刹车）

H 桥 (H-Bridge) 原理

H 桥由 4 个开关 (MOS 管) 组成, 可控制电流方向

四种工作状态

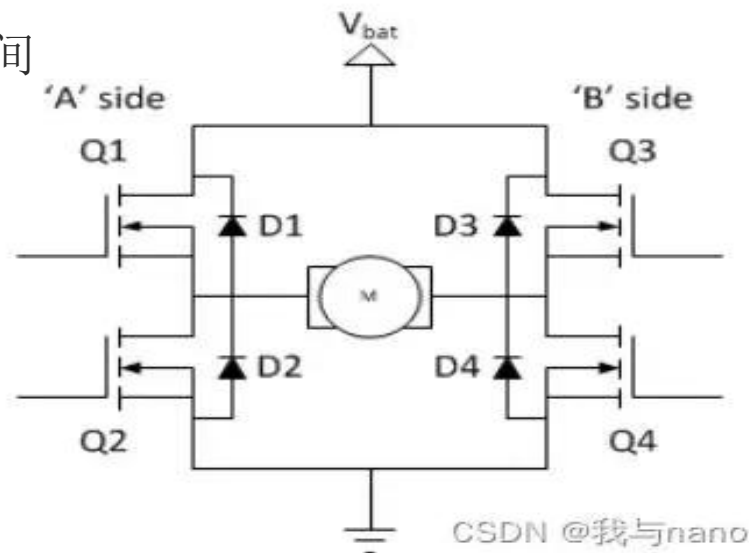
正转: Q1+Q4 闭合, Q2+Q3 断开 → 电流从左到右流过电机

反转: Q2+Q3 闭合, Q1+Q4 断开 → 电流从右到左流过电机

制动: Q2+Q4 闭合 (下桥臂短路) → 电机快速停止

滑行: 全部断开 → 电机自由惯性转动

死区: 换向时两个开关都处于关断状态的空白时间



占空比为什么能控制转速？

ENA 接的 PWM 信号，本质是让 H 桥按这个节奏 “通-断-通-断”

电机两端真实看到的不是稳定直流电压，而是在 0V 和电源电压（VS）之间跳变的方波

电机线圈的电感 + 转子的机械惯量，跟不上 PWM 这么快的开关节奏，相当于自带 “低通滤波” 效果

电机实际感受到的是这个方波的平均电压：

$$\text{平均电压} \approx \text{占空比} \times VS$$

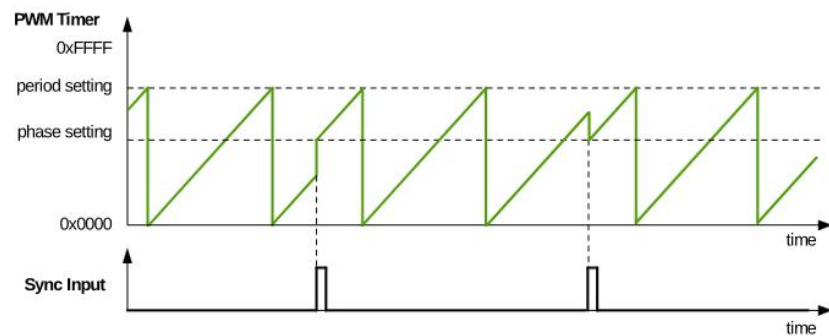


图 36.3-6. 递增计数模式波形

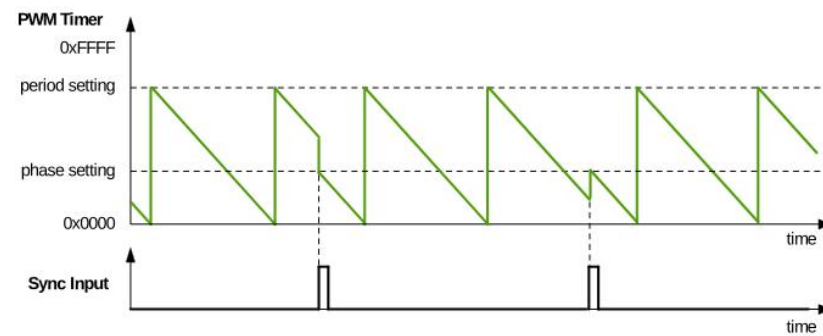


图 36.3-7. 递减计数模式波形

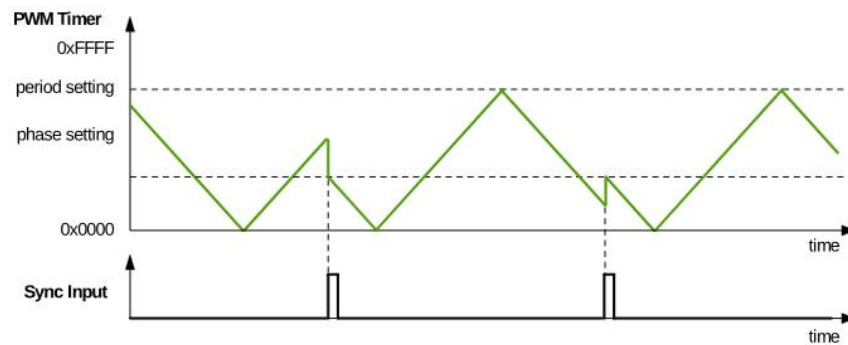


图 36.3-8. 递增递减循环模式波形，同步事件后递减

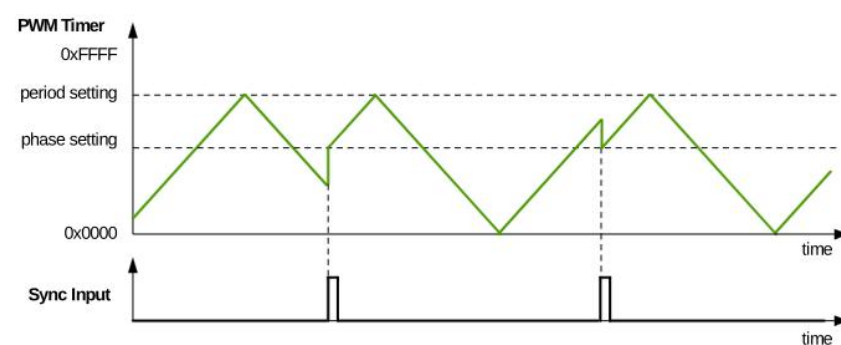
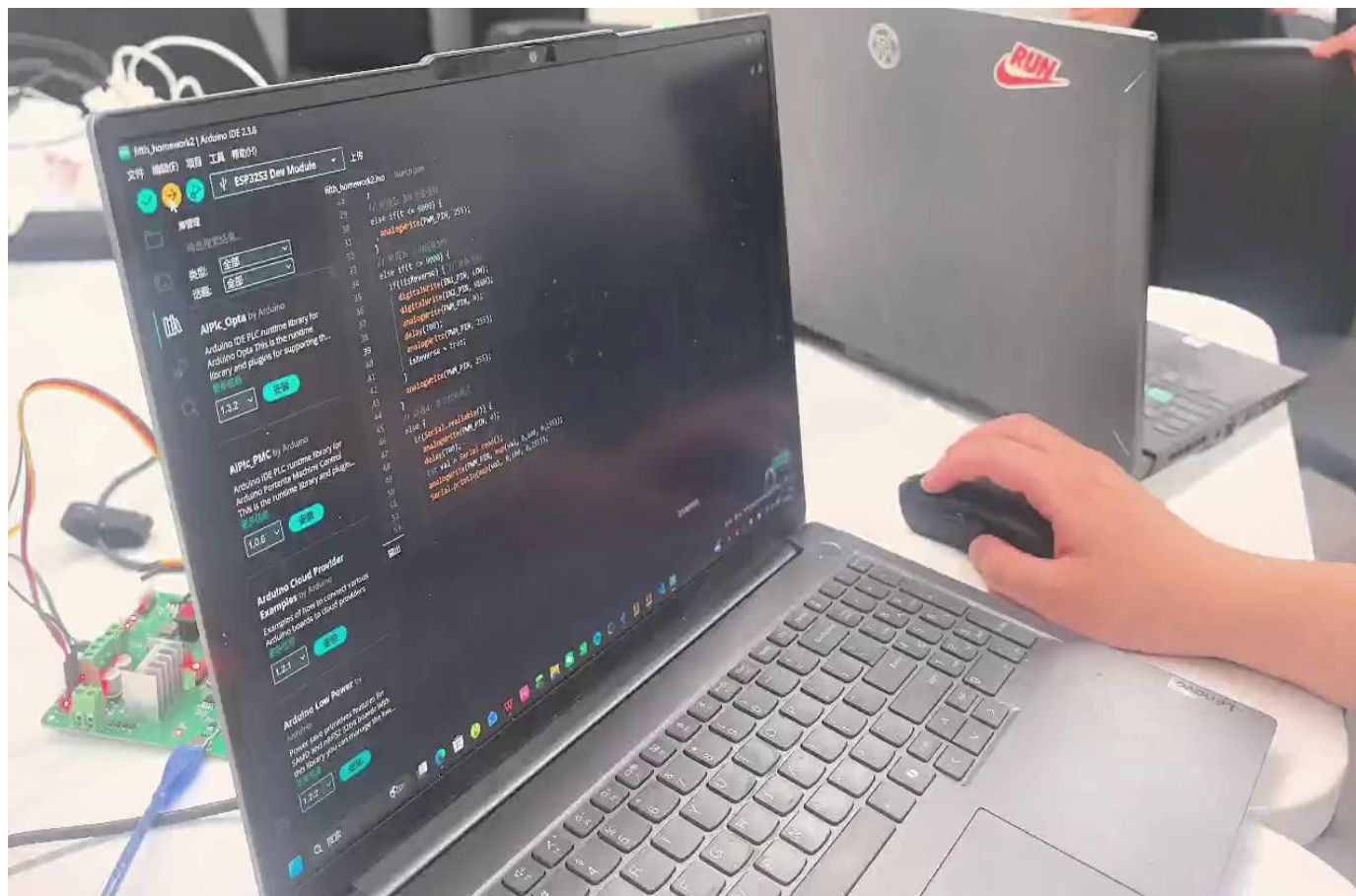


图 36.3-9. 递增递减循环模式波形，同步事件后递增



实验演示



07 安全指南 —— 防烧毁



接线铁律

1. 信号分离

单片机只输出逻辑信号（GPIO 管方向，PWM 管速度），给驱动板控制引脚（IN1, IN2, ENA）

2. 电源隔离

VCC → 接 ESP32-S3 的 3.3V（给驱动板芯片供电）

VIN → 接外部独立电池组或市电插头（7.4V / 12V，专门给电机供电）

3. 必须共地 (GND)

ESP32-S3 的 GND ↔ 外部电池负极 ↔ 驱动板 GND → 必须连在一起！

不共地 → 信号没有参考基准 → 电机绝对不转！

【附录/参考】LEDC 寄存器表

以下信息来自 ESP32-S3 TRM 第 35 章，仅供查阅，可能有误，一切以数据手册为准

功能	寄存器名	关键作用
定时器分频系数	LEDC_CLK_DIV_TIMERx	= A + B/256，支持小数分频
计数器位宽	LEDC_TIMERx_DUTY_RES	0~14，决定 PWM 频率上限
应用新定时器配置	LEDC_TIMERx_PARA_UP	写1生效（溢出时自动更新）
当前计数值	LEDC_TIMERx_CNT	只读
选择时钟源	LEDC_APB_CLK_SEL	1=APB / 2=RC_FAST / 3=XTAL
通道选择定时器	LEDC_TIMER_SEL_CHn	0~3
使能 PWM 输出	LEDC_SIG_OUT_EN_CHn	1=开启输出
高电平触发点	LEDC_HPOINT_CHn	CNT==HPoint 时输出变高
占空比设置	LEDC_DUTY_CHn	LPoint = HPoint + DUTY
占空比渐变使能	LEDC_DUTY_START_CHn	1=开启硬件自动渐变
渐变步长	LEDC_DUTY_SCALE_CHn	每次变化量
渐变速度	LEDC_DUTY_CYCLE_CHn	多少周期变化一次
渐变总步数	LEDC_DUTY_NUM_CHn	渐变停止前变化次数

频率公式：f_PWM = f_LEDC_CLK / (LEDC_CLK_DIV × 2^DUTY_RES)

基地址 0x6001_9000，各寄存器偏移见 TRM §35.3

谢谢大家

